

2Chance Web Application

Sustainable Software Engineering Project Report

Aldo Manco

23 January 2026

Contents

1	Panoramica dell'Architettura del Software	3
1.1	Analisi dei Componenti dell'Architettura a Livelli	3
1.1.1	Il Livello View (Presentazione e Interazione)	3
1.1.2	Il Livello Controller (Coordinamento degli Input e Routing)	3
1.1.3	Il Livello Service (Orchestrazione della Logica di Business)	4
1.1.4	Il Livello Data Access (Persistenza e DAO)	4
1.1.5	I Model Bean (Rappresentazione dei Dati)	4
1.2	Comportamento Dinamico: Il Flusso di Autenticazione	5
1.3	Miglioramenti di Dependability e Toolchain	5
1.3.1	Testing e Analisi della Copertura (JUnit, Mockito, JaCoCo e Codecov)	6
1.3.2	Specifiche e Verifica Formale (JML e OpenJML)	6
1.3.3	Mutation Testing (PITest)	6
1.3.4	Containerizzazione e Orchestrazione (Docker e Docker Compose)	6
1.3.5	Sicurezza e Valutazione delle Vulnerabilità	6
2	Sostenibilità Energetica del Back-End	8
2.1	Analisi Statica del Codice (Creedengo)	8
2.1.1	Configurazione di SonarQube con il Plugin Creedengo	8
2.1.2	Selezione delle Regole di Eco-Design	9
2.1.3	Output dell'Analisi Statica e Piano di Refactoring	9
2.1.4	Unit e Integration Testing Iterativo	10
2.1.5	Validazione dell'Analisi Statica (Creedengo Post-Refactoring)	11
2.2	Validazione delle Ottimizzazioni: Baseline vs SSE-Improvements	11
2.2.1	Introduzione a EnergiBridge per la Profilazione	11
2.2.2	Microbenchmarking Energetico sui DAO (JMH + EnergiBridge)	11
2.2.3	System Level Performance Testing (JMeter + EnergiBridge)	13
2.2.4	Automazione, Isolamento e Sincronizzazione dei Test	17
3	Valutazione e Ottimizzazione Ambientale del Livello Front-End	23
3.1	Analisi Ambientale del Baseline (GreenIT ed EcoIndex)	23
3.1.1	Metodologia di Valutazione: Contesto Reale (GreenIT-Analysis) vs Contesto Laboratorio (EcoIndex)	24
3.2	Interventi di Refactoring e Ottimizzazione Ambientale	25
3.3	Validazione dell'Impatto del Refactoring	28
3.3.1	Quantificazione dell'Ottimizzazione Globale	30
4	Monitoraggio Licenze Open Source	32
4.1	Analisi della Compliance e dei Rischi Legali (FOSSA)	32
4.1.1	Sostenibilità Legale ed Economica: Software Composition Analysis e Conformità delle Licenze	32
4.1.2	Monitoraggio delle Licenze Open Source tramite FOSSA	32

4.1.3	Esecuzione dell'Analisi FOSSA	33
4.1.4	Albero delle Dipendenze e Inventario delle Licenze	34
4.2	Integrazione Cloud e Continuous Compliance	36
4.2.1	Risultati della Scansione (Policy Scan) e Valutazione del Rischio Legale per Macro-Aree Architetture	37
4.2.2	Generazione della Software Bill of Materials (SBOM)	40
4.3	Integrazione di FOSSA nel pipeline CI/CD	40
4.3.1	Automazione degli Obblighi di Attribuzione	40

Chapter 1

Panoramica dell'Architettura del Software

1.1 Analisi dei Componenti dell'Architettura a Livelli

La piattaforma *2Chance* si basa su un'architettura modulare a livelli che segue il tradizionale paradigma Model–View–Controller (MVC). Di conseguenza, le responsabilità di sistema sono rigorosamente mappate: la presentazione dei dati è gestita dalla View (JavaServer Pages e JavaScript); il coordinamento degli input è gestito dal Controller; e il Model è suddiviso in Service (logica di business), Data Access Object (persistenza) e Bean (strutture dati). Questo rigoroso disaccoppiamento costituisce la base esistente su cui sono stati applicati i miglioramenti di dependability del progetto.

1.1.1 Il Livello View (Presentazione e Interazione)

Componente: pacchetto `webapp` (es. `index.jsp`, `login.jsp`).

Il livello View costituisce la logica di presentazione ed è responsabile del rendering dell'interfaccia utente (UI) e dell'acquisizione degli input dell'utente. È implementato utilizzando i linguaggi standard del web, tra cui HyperText Markup Language (HTML), Cascading Style Sheets (CSS) per lo stile e JavaScript (JS) per l'interattività lato client. Utilizzando JavaServer Pages (JSP), visualizza dinamicamente i dati forniti dal Controller, tipicamente passati tramite attributi di request o di sessione.

Sebbene la View avvii il flusso di dati trasmettendo i parametri dell'utente tramite richieste HTTP, le è architettonicamente proibito eseguire logica di business o accedere direttamente al livello di persistenza.

1.1.2 Il Livello Controller (Coordinamento degli Input e Routing)

Componente: pacchetto `controller` (es. `LoginServlet`, `RegistrazioneServlet`).

Il Controller funge da punto di ingresso centralizzato per l'elaborazione lato server. Implementato tramite Java Servlet, intercetta e gestisce le richieste HTTP in ingresso (nello specifico, GET e POST) trasmesse dal client. Agisce come direttore del traffico dell'applicazione, assicurando che le azioni dell'utente vengano instradate agli appropriati gestori della logica di business. Il Controller agisce strettamente come intermediario e non contiene logica di business.

Il suo flusso di lavoro operativo consiste in tre fasi distinte:

1. **Parsing:** estrae i parametri di input dall'oggetto request e gestisce la sessione HTTP per preservare lo stato dell'utente attraverso interazioni stateless.

2. **Delegation (Delega):** invoca i metodi appropriati all'interno del livello Service, passando solo gli argomenti necessari.
3. **Routing (Instradamento):** al completamento dell'esecuzione del servizio, determina la strategia di risposta, inoltrando (forward) il contesto della richiesta a una vista JSP per il rendering o reindirizzando (redirect) l'utente verso una nuova risorsa per prevenire il reinvio del modulo.

1.1.3 Il Livello Service (Orchestrazione della Logica di Business)

Componente: pacchetto `services` (es. `LoginService`, `AdminService`).

Il livello Service costituisce il nucleo operativo dell'applicazione, incapsulando la logica di business del sistema. Serve come astrazione che disaccoppia la gestione degli input (Controller) dall'archiviazione dei dati (DAO). Isolando queste responsabilità, l'architettura centralizza le regole di business e le rende indipendenti dall'interfaccia utente e dalla tecnologia del database.

Implementati come classi Java standard, i componenti Service orchestrano flussi di lavoro complessi e fanno rispettare gli invarianti di dominio. In particolare, un Service convalida gli input rispetto alle regole specifiche dell'applicazione (es. assicurando che una password soddisfi i criteri di complessità o verificando la disponibilità in magazzino) prima di intraprendere qualsiasi azione. Per preservare la purezza architetturale, al livello Service è proibito eseguire direttamente query SQL. Invece, coordina la persistenza invocando il livello Data Access per recuperare o persistere i dati, garantendo che l'elaborazione logica rimanga distinta dalla manipolazione fisica dei dati.

1.1.4 Il Livello Data Access (Persistenza e DAO)

Componente: pacchetto `model.dao` (es. `UtenteDAO`, `ProdottoDAO`).

Il livello Data Access implementa il pattern Data Access Object (DAO), che astrae e incapsula tutto l'accesso alla fonte dei dati. Questo design assicura che il resto dell'applicazione rimanga agnostico rispetto all'implementazione concreta del database, consentendo potenziali modifiche nel meccanismo di archiviazione senza avere un impatto sulla logica di livello superiore. Questo livello è l'unica autorità autorizzata a eseguire comandi SQL.

Utilizzando JDBC (Java Database Connectivity) e un connection pool di Tomcat per un'efficiente gestione delle risorse, i DAO gestiscono la comunicazione a basso livello con il database. I loro compiti principali includono:

1. **Operazioni CRUD:** esecuzione di transazioni Create, Read, Update e Delete.
2. **Object-Relational Mapping (ORM):** trasformazione manuale dei dati relazionali grezzi (es. righe di `ResultSet`) in oggetti Java ad alto livello (Model Bean) e viceversa. Questa traduzione consente al livello Service di manipolare oggetti Java anziché tipi di dati specifici del database.

1.1.5 I Model Bean (Rappresentazione dei Dati)

Componente: pacchetto `model.beans` (es. `Utente`, `Prodotto`).

I Model Bean rappresentano le entità funzionali fondamentali del sistema. Strutturalmente, sono implementati come Plain Old Java Object (POJO). La loro responsabilità principale è

l'incapsulamento dei dati, consentendo il trasporto dello stato attraverso i livelli View, Controller, Service e Data Access.

Ogni Bean rispecchia una specifica entità nel modello di dominio, corrispondendo tipicamente a una tabella nel database relazionale. I Bean mantengono lo stato attraverso attributi privati, ai quali si accede e che si modificano esclusivamente tramite metodi pubblici di accesso (getter) e mutazione (setter).

1.2 Comportamento Dinamico: Il Flusso di Autenticazione

Per illustrare l'interazione dinamica tra i componenti architetturali, esaminiamo il flusso di controllo di una procedura standard di autenticazione utente (login). Questa sequenza mostra come i dati si propagano dall'interfaccia utente al database e viceversa, rispettando la separazione delle responsabilità (separation of concerns) definita sopra:

1. **Interazione e richiesta (livello View):** l'utente inserisce le credenziali in `login.jsp`. All'invio, il browser genera una richiesta HTTP POST contenente i parametri (email e password) e la trasmette al server.
2. **Intercettazione e delega (livello Controller):** la `LoginServlet` intercetta la richiesta e ne analizza il corpo per estrarre i parametri di input. In conformità con il suo ruolo di coordinamento, non esegue alcuna verifica dei dati; al contrario, delega l'operazione invocando `login()` su `LoginService`.
3. **Orchestrazione e validazione (livello Service):** `LoginService` riceve i parametri grezzi ed esegue una validazione preliminare (es. assicurandosi che i campi non siano vuoti). Se la validazione ha esito positivo, richiede il recupero dei dati chiamando `getUserByEmailPassword()` su `UtenteDAO`.
4. **Esecuzione della query (livello Data Access):** `UtenteDAO` stabilisce una connessione al database, costruisce ed esegue una query `SELECT` sicura. Questo è l'unico punto nel flusso in cui l'applicazione comunica direttamente con l'archiviazione di persistenza.
5. **Mappatura degli oggetti (livello Data Access):** dal `ResultSet` restituito, il DAO esegue il mapping oggetto-relazionale: istanzia un nuovo Bean `Utente`, popola i suoi campi con i valori del database e restituisce l'oggetto al livello Service.
6. **Gestione dello stato e risposta (livello Controller):** il Service restituisce il Bean `Utente` popolato al Controller. Il Controller quindi (i) persiste lo stato memorizzando il Bean nella sessione HTTP, effettuando di fatto l'accesso dell'utente, e (ii) esegue la navigazione inviando una risposta di reindirizzamento che istruisce il client a caricare `dashboard.jsp`, dove viene visualizzato il nome dell'utente.

1.3 Miglioramenti di Dependability e Toolchain

In seguito allo sviluppo iniziale, l'architettura è stata significativamente modernizzata durante il progetto di software dependability. Questa fase ha introdotto diverse tecnologie e metodologie avanzate per garantire la qualità del codice, la sicurezza e la manutenibilità.

1.3.1 Testing e Analisi della Copertura (JUnit, Mockito, JaCoCo e Codecov)

È stata implementata una metodologia di testing completa e bottom-up per convalidare i componenti del sistema. **JUnit 5** funge da framework principale per i test automatizzati, mentre **Mockito** è ampiamente utilizzato per creare mock delle dipendenze esterne, come le connessioni al database nei DAO e le richieste HTTP nei Controller, consentendo unit test isolati. Inoltre, **JaCoCo** è integrato per misurare la copertura del codice (es. line e branch coverage), e i risultati vengono aggregati e visualizzati tramite **Codecov** durante la pipeline di Continuous Integration (CI), garantendo che gli standard di test siano continuamente rispettati.

1.3.2 Specifica e Verifica Formale (JML e OpenJML)

Per elevare l'affidabilità delle classi critiche, sono stati adottati metodi formali. Il **Java Modeling Language (JML)** è utilizzato per annotare il codice Java con specifiche formali, definendo precondizioni, postcondizioni e invarianti di classe. **OpenJML** viene quindi impiegato come strumento di verifica statica per dimostrare matematicamente che l'implementazione Java soddisfi rigorosamente tali specifiche, prevenendo errori logici e fallimenti in casi limite (edge-case).

1.3.3 Mutation Testing (PITest)

Per valutare l'effettiva capacità di rilevamento dei guasti delle suite di test, è stato introdotto il mutation testing tramite **PITest**. Invece di fare affidamento esclusivamente sulla line coverage, PITest inietta automaticamente piccoli guasti sintattici (mutanti) nel codice dell'applicazione ed esegue la suite di test. Se i test falliscono (uccidendo il mutante), ciò verifica che i test sono sufficientemente robusti per individuare le regressioni.

1.3.4 Containerizzazione e Orchestrazione (Docker e Docker Compose)

Per eliminare il problema del “funziona sulla mia macchina” e semplificare i deployment, l'infrastruttura dell'applicazione è stata containerizzata utilizzando **Docker**. **Docker Compose** è utilizzato per orchestrare l'ambiente multi-container, definendo servizi interconnessi distinti per l'applicazione web, il database MySQL e le utilità di testing. Questa configurazione garantisce un ambiente di esecuzione coerente e riproducibile attraverso lo sviluppo, le pipeline CI/CD e la produzione.

1.3.5 Sicurezza e Valutazione delle Vulnerabilità

È stata stabilita una solida pipeline di sicurezza automatizzata all'interno delle GitHub Actions per monitorare continuamente il repository e i suoi artefatti:

- **GitGuardian (Secret Detection):** Implementato per la scansione automatizzata dei secret, analizza i commit per rilevare e prevenire l'esposizione di password codificate, chiavi API o token sensibili.
- **Snyk (Software Composition Analysis):** Impiegato per la SCA, scansiona attivamente le dipendenze Maven per identificare e mitigare le vulnerabilità di sicurezza note (CVE) nelle librerie di terze parti.

- **SonarQube Cloud (Static Analysis):** Utilizzato per il Static Application Security Testing (SAST), esegue una profonda analisi statica sulla base di codice per rilevare code smell, debito tecnico e vulnerabilità di sicurezza, in particolare all'interno del livello Controller.

Chapter 2

Sostenibilità Energetica del Back-End

Il Back-End di 2Chance gestisce la logica del sistema, le interrogazioni al database e la gestione delle interfacce. Essendo il componente che compie il maggior sforzo computazionale, è anche quello che consuma più energia. Per ottimizzarlo, il mio lavoro si articola in tre punti: individuare tramite analisi statica e revisione del codice le porzioni di codice meno efficienti, procedere al refactoring per ridurre lo spreco di risorse e misurare il consumo energetico effettivo durante l'uso prima e dopo il refactoring.

2.1 Analisi Statica del Codice (Creedengo)

Per l'Analisi Statica dell'Eco-Design basata su regole (*Rule-Based Eco-Design Static Analysis*), è necessario integrare il codice sorgente locale, un'istanza attiva di SonarQube e il plugin Creedengo. Poiché Creedengo opera come plugin per SonarQube, esso si integra direttamente nella pipeline di Static Application Security Testing (SAST) esistente. Dato che il progetto è containerizzato tramite Docker, l'esecuzione locale di SonarQube in parallelo all'applicazione risulta estremamente efficiente. Di seguito viene descritta la guida pratica passo-passo per la configurazione, l'esecuzione e l'estrazione delle eco-vulnerabilità rilevate.

2.1.1 Configurazione di SonarQube con il Plugin Creedengo

Per agevolare l'installazione e la configurazione di SonarQube con Creedengo su qualsiasi altra infrastruttura, lo script di automazione orchestra l'intero ciclo di deployment dell'ambiente di analisi statica attraverso i seguenti passaggi sequenziali:

1. **Validazione dell'Ambiente:** Verifica della disponibilità e dell'esecuzione attiva del Docker daemon sul sistema host.
2. **Approvvigionamento del Plugin:** Controllo della presenza locale del plugin Creedengo per Java. In caso di assenza, lo script ne effettua il download dinamico dalle release ufficiali del repository GitHub.
3. **Provisioning di SonarQube:** Configurazione e avvio di un'istanza standalone in un docker container di SonarQube, con l'esposizione delle interfacce di rete necessarie.
4. **Iniezione della Libreria:** Trasferimento del plugin scaricato all'interno del file system isolato del container di SonarQube, inserendolo nel percorso di sistema riservato alle estensioni.

5. **Riavvio ed Inizializzazione:** Invio di un segnale di riavvio al container per forzare il caricamento a runtime del nuovo componente e registrare nel database le regole di eco-design.

2.1.2 Selezione delle Regole di Eco-Design

Per massimizzare il ritorno ecologico delle attività di refactoring e ridurre il rumore nelle metriche di telemetria statica, sono state selezionate inizialmente 11 regole fondamentali del ruleset Creedengo Java. Ciascuna di queste regole è stata scelta in base alla sua pertinenza tecnologica con l'architettura MVC di 2Chance, un'applicazione web monolitica basata esclusivamente su Java Servlets e interazioni dirette con il database tramite pattern JDBC/DAO nativi (senza l'uso di framework pesanti come Spring).

2.1.3 Output dell'Analisi Statica e Piano di Refactoring

Dopo aver configurato l'ambiente di analisi statica ed eseguito la scansione iniziale (fase di *baseline*), sono state rilevate complessivamente **64 violazioni di eco-design** distribuite unicamente su 4 delle 11 regole principali del ruleset Creedengo che rappresentano anti-pattern strutturali.

Questi problemi interessavano in modo critico i Model Data Access Object. Nell'ecosistema dell'architettura software di 2Chance, infatti, il livello di accesso ai dati (DAO) rappresenta il principale *bottleneck* dal punto di vista del consumo energetico. A differenza dei Controller o dei Service, i quali si limitano all'orchestrazione di logiche prettamente in-memory, i DAO si fanno carico della comunicazione inter-processo con il database relazionale.

Per risolvere i problemi architetturali emersi durante la revisione del codice, sanare le violazioni critiche individuate dall'analisi statica di Creedengo e massimizzare l'efficienza ecologica, in particolar modo a livello di persistenza, si è deciso di procedere a un refactoring profondo di tutti i componenti del sistema, intervenendo sistematicamente sui Data Access Object (DAO) e sui Model Beans.

Di conseguenza, vengono di seguito approfondite esclusivamente le regole realmente coinvolte nelle vulnerabilità riscontrate, integrando per ciascuna di esse sia le inefficienze strutturali scoperte durante la revisione del codice (**il Problema**) sia le relative soluzioni architetturali adottate (**la Soluzione**):

1. Avoid getting the size of the collection in the loop (GCI3):

- **Problema:** Questa regola rappresenta una variante diretta delle specifiche di progetto relative all'efficienza computazionale. L'invocazione continua del metodo `.size()` all'interno della guardia di un ciclo forza la JVM a ricalcolare i limiti della collezione a ogni singola iterazione, spreco di cicli di clock inutili.
- **Soluzione:** Per prevenire interrogazioni continue ad oggetti nell'area Heap, le invocazioni di metodi all'interno dei parametri di guardia dei cicli sono state estratte introducendo variabili primitive locali prima del ciclo. Questo semplice *caching* permette alla CPU di memorizzare il valore limite in un registro fisico, abilitando ottimizzazioni Just-In-Time (JIT) e annullando gli sprechi computazionali.

2. Avoid SQL request in loop (GCI72):

- **Problema:** L'esecuzione di interrogazioni al database all'interno di cicli iterativi è un fattore altamente inefficiente. Genera un overhead continuo di rete e costringe il motore del database a un lavoro sequenziale pesante, mantenendo attivi i thread

di Tomcat ed impedendo alla CPU di scendere in modalità a basso consumo (C-states). L'analisi di questa violazione ha portato alla luce due inefficienze critiche nei DAO: l'assenza di batch processing nelle scritture iterative (come in `ProdottoDAO` e `OrdineDAO`) e il problema della moltiplicazione delle query N+1 per le estrazioni collegate (come in `OrdineDAO.getOrdiniByUtente`).

- **Soluzione:** Le sequenze di INSERT o UPDATE eseguite all'interno di cicli iterativi sono state sostituite con operazioni batch native del driver JDBC (`addBatch()` e `executeBatch()`). Il raggruppamento delle scritture in un'unica transazione ha minimizzato il tempo di acquisizione delle connessioni, prevenendo la saturazione del pool. Inoltre, per il problema del querying N+1, l'estrazione iterativa è stata consolidata in un'unica query aggregata tramite costrutti LEFT JOIN, delegando l'onere dell'assemblaggio delle entità a una `LinkedHashMap` in memoria e abbattendo i roundtrip con il database.

3. Do not call a function when declaring a for-type loop (GCI69):

- **Problema:** Classificato a priorità elevata, questo fault identifica chiamate a funzioni invarianti all'interno dei parametri di inizializzazione o di guardia dei cicli, causando un inutile dispendio di CPU a causa delle continue rivalutazioni.
- **Soluzione:** Analogamente alla regola GCI3, risolvere questo anti-pattern ha richiesto l'estrazione del risultato delle chiamate a funzione in variabili primitive locali prima dell'esecuzione del ciclo. In questo modo si evitano migliaia di invocazioni ridondanti del medesimo metodo, consentendo alla JVM di ottimizzare l'esecuzione del blocco iterativo.

4. Don't use the query SELECT * FROM (GCI74):

- **Problema:** La presenza di questa regola nei moduli di persistenza ha messo in luce l'Anti-Pattern "SELECT *" e il Trasferimento Dati Incontrollato: le query di recupero facevano un uso sistematico dell'operatore SELECT *. Questo approccio costringeva il database engine a eseguire operazioni di I/O su disco per estrarre l'interezza dei campi, compresi quelli non necessari o molto pesanti (come le stringhe *Base64* delle immagini), per poi serializzarli sulla rete, costringendo la JVM ad allocare massicciamente oggetti temporanei in memoria.
- **Soluzione:** Tutte le query di lettura indiscriminate nei DAO sono state riscritte esplicitando unicamente le colonne necessarie nella proiezione SQL. Riducendo il payload di rete trasferito, si è alleggerita la fase di *Object-Relational Mapping* (ORM), diminuendo le allocazioni temporanee nella memoria Heap della JVM e, di conseguenza, lo stress energetico dovuto ai cicli del *Garbage Collector*. Parallelamente, il mapping dei risultati è stato allineato ai nomi simbolici delle colonne, aumentando la resilienza dello schema.

2.1.4 Unit e Integration Testing Iterativo

Ogni singola classe dell'applicazione è stata rifattorizzata singolarmente, seguendo un approccio incrementale e rigoroso. Al termine delle modifiche di ciascuna classe (Bean o DAO), è stata eseguita tramite Maven e JUnit 5 la specifica test suite di *unit testing* relativa al solo componente modificato, in modo da isolare e correggere eventuali errori semantici o disallineamenti dei mock (ad esempio per l'uso di query batch e nomi di colonne al posto degli indici). Una volta validato con successo il componente isolato, si è proceduto all'esecuzione di una fase di *integration testing* in cui è stata lanciata l'intera suite globale composta da oltre 500 test.

Questo processo di verifica a doppio passaggio è stato fatto per ogni singola classe, garantendo la totale assenza di regressioni e certificando che la logica di business dell'applicazione 2Chance rimanesse del tutto inalterata prima di validare le ottimizzazioni in termini energetici.

2.1.5 Validazione dell'Analisi Statica (Creedengo Post-Refactoring)

In seguito alle attività di refactoring orientate alla sostenibilità energetica, è stata eseguita una nuova sessione di scansione con Creedengo su SonarQube. I risultati aggiornati confermano che **tutte le 64 eco-vulnerabilità sono state risolte ed i relativi fault sono stati chiusi con successo (stato CLOSED/FIXED)**. La completa risoluzione di questi fault garantisce che l'applicazione non presenti inefficienze strutturali latenti a livello di CPU-bound e I/O-bound, allineandosi con i requisiti di sostenibilità energetica previsti per il sistema 2Chance.

2.2 Validazione delle Ottimizzazioni: Baseline vs SSE-Improvements

Al fine di validare l'efficacia del refactoring, il capitolo si articola ora nel confronto diretto tra la baseline e il branch `sse-improvements`. Verranno esplorati prima i test di microbenchmarking isolati tramite JMH ed EnergiBridge, seguiti dal System Level Performance Testing tramite JMeter ed EnergiBridge.

2.2.1 Introduzione a EnergiBridge per la Profilazione

Per la raccolta di metriche empiriche sul consumo energetico, il framework metodologico prevede l'integrazione di **EnergiBridge**. Tale strumento, orientato alla profilazione *cross-platform*, monitora l'impatto energetico delle applicazioni interfacciandosi direttamente con i sensori hardware di architetture quali x86, AMD e Apple Silicon. Questa compatibilità risulta essenziale per il progetto 2Chance, le cui misurazioni saranno effettuate su piattaforma AMD Ryzen AI 9 HX 370. Lo strumento opera tramite interfaccia a riga di comando, campionando il consumo energetico a intervalli regolari durante l'esecuzione di un processo *target* ed esportando i dati aggregati in formato CSV.

Per installare e configurare EnergiBridge su Windows, ho creato la directory di sistema `C:\Program Files\EnergiBridge`. Ho quindi scaricato l'ultima versione stabile, la 0.0.7, dal repository ufficiale GitHub ([tdurieux/EnergiBridge](https://github.com/tdurieux/EnergiBridge)) e ho integrato `LibreHardwareMonitor`, un plugin necessario per permettere al software di leggere i dati dai registri della CPU su Windows.

2.2.2 Microbenchmarking Energetico sui DAO (JMH + EnergiBridge)

Per convalidare sperimentalmente a livello di singolo metodo le inefficienze architetturali emerse dalla revisione del codice e dalle violazioni strutturali evidenziate dall'analisi statica (*Creedengo*), sono emersi specifici anti-pattern architetturali concentrati in particolare in 3 classi DAO: `ProdottoDAO`, `RecensioneDAO` e `OrdineDAO`. A causa di questi anti-pattern che causavano un elevato footprint energetico nella versione di *baseline*, queste 3 classi sono state scelte per effettuare una fase di microbenchmarking per tutti i loro metodi. L'isolamento di queste porzioni critiche di codice costituisce infatti il prerequisito fondamentale per il microbenchmarking effettuato con il framework JMH combinato con la profilazione di EnergiBridge, consentendo di misurare con estrema precisione le performance e l'assorbimento energetico (in Joule per operazione) prima e dopo il refactoring.

Strumento di Profilazione Energetica Co-Sincronizzato

Per combinare la precisione temporale del microbenchmarking con la profilazione energetica dell'hardware, ho implementato uno script Python (`run_jmh_energy_profiler.py`) che coordina l'esecuzione dei test JMH e l'acquisizione dei dati di EnergiBridge. Lo script compila prima il codice Java con Maven e individua tutti i metodi di benchmark disponibili nelle classi della directory fornita in input. Successivamente, per evitare interferenze tra i test, esegue i benchmark JMH in modalità isolata per ciascun metodo per evitare il fenomeno dell'inquinamento della memoria Heap e del cross-talk tra thread differenti. Ciascun processo EnergiBridge avvolge un'istanza JVM pulita, garantendo che le metriche di consumo non risentano dell'overhead di esecuzioni precedenti. Per eliminare il rumore e le anomalie di lettura tipiche dei registri hardware RAPL, i dati energetici vengono filtrati applicando le condizioni di validità energetica rispetto al tempo (Δt): $0 \leq \Delta E_{\text{CPU}} \leq 55,0 \times \Delta t$ e $0 \leq \Delta E_{\text{Core}} \leq 45,0 \times \Delta t$. Infine, lo script unisce le metriche temporali di JMH con quelle energetiche purificate di EnergiBridge, producendo un report finale che riassume il consumo energetico totale (J), la potenza media (W), il carico e la frequenza della CPU, e i picchi di memoria RAM allocata.

I test sono stati eseguiti impostando una configurazione a thread singolo (`@State(Scope.Thread)`), con 5 iterazioni di warmup e 5 iterazioni di misurazione per ciascun modulo isolato. Le metriche temporali fornite da JMH sono espresse in millisecondi per operazione (ms/op), mentre le metriche di consumo energetico fornite da EnergiBridge sono misurate in Joule (J).

Confronto dei Risultati dei Benchmark (Baseline vs Post-Refactoring)

Al fine di verificare quantitativamente il guadagno prestazionale ed energetico ottenuto in seguito alle attività di refactoring, la suite è stata eseguita su entrambi i branch (`main` per la baseline e `sse-improvements` per la versione ottimizzata). Nelle tabelle seguenti viene presentato il confronto dettagliato delle metriche reali per ciascun DAO, evidenziando le variazioni percentuali del refactoring.

Metodo di Benchmark	Tempo Base (ms)	Tempo Post (ms)	Var. Tempo	Energia Base (J)	Energia Post (J)	Var. Energia
benchmarkAggiungiSpecifiche	23,6479	12,7803	-45,96%	74,72	80,41	+7,62%
benchmarkCercaProdottiInvalid	0,0008	0,0008	+0,00%	90,06	85,15	-5,45%
benchmarkCercaProdottiValid	1,2442	1,0937	-12,10%	74,99	77,76	+3,69%
benchmarkGetProdotti	1,4448	1,1194	-22,52%	279,08	283,47	+1,57%
benchmarkGetProdottiByCategoriaInvalid	0,0008	0,0007	-12,50%	84,19	79,16	-5,97%
benchmarkGetProdottiByCategoriaNonExistent	1,1278	1,0905	-3,31%	76,26	75,45	-1,06%
benchmarkGetProdottiByCategoriaValid	1,2043	1,1371	-5,58%	77,42	76,68	-0,96%
benchmarkGetProdottoByIdInvalid	0,0009	0,0009	+0,00%	82,91	86,52	+4,35%
benchmarkGetProdottoByIdNonExistent	1,0384	1,0238	-1,41%	76,03	78,59	+3,37%
benchmarkGetProdottoByIdValid	19,1854	3,8568	-79,90%	81,32	93,89	+15,46%
benchmarkGetUltimiProdotti	1,1066	1,1989	+8,34%	87,18	75,35	-13,57%

Table 2.1: Confronto temporale ed energetico per la classe ProdottoDAOBenchmark.

Metodo di Benchmark	Tempo Base (ms)	Tempo Post (ms)	Var. Tempo	Energia Base (J)	Energia Post (J)	Var. Energia
benchmarkAddRecensione	7,5219	7,9374	+5,52%	77,45	84,09	+8,57%
benchmarkGetRecensioniByProdottoInvalid	0,0008	0,0008	+0,00%	73,85	78,15	+5,82%
benchmarkGetRecensioniByProdottoNonExistent	1,1259	0,9866	-12,37%	75,78	77,84	+2,72%
benchmarkGetRecensioniByProdottoValid	15,3983	1,0864	-92,94%	76,26	77,56	+1,70%
benchmarkGetRecensioniByUtenteInvalid	0,0008	0,0009	+12,50%	82,31	81,80	-0,62%
benchmarkGetRecensioniByUtenteNonExistent	1,0402	1,0266	-1,31%	95,78	76,63	-19,99%
benchmarkGetRecensioniByUtenteValid	15,0104	1,0249	-93,17%	77,52	73,83	-4,76%

Table 2.2: Confronto temporale ed energetico per la classe RecensioneDAOBenchmark.

Metodo di Benchmark	Tempo Base (ms)	Tempo Post (ms)	Var. Tempo	Energia Base (J)	Energia Post (J)	Var. Energia
benchmarkAddOrdineValid	13,5605	13,1136	-3,30%	90,12	87,14	-3,31%
benchmarkCarrelloOperations	0,0000	0,0000	N/A	88,49	84,40	-4,62%
benchmarkGetOrdineByIdInvalid	0,0009	0,0007	-22,22%	93,15	87,30	-6,28%
benchmarkGetOrdineByIdValid	1,0374	1,0376	+0,02%	93,99	86,78	-7,67%
benchmarkGetOrdiniByUtenteInvalid	0,0009	0,0008	-11,11%	100,47	93,48	-6,96%
benchmarkGetOrdiniByUtenteValid	41,3830	1,0195	-97,54%	88,68	96,11	+8,38%
benchmarkGetProdottoOrdineInvalid	0,0009	0,0008	-11,11%	93,50	88,88	-4,94%
benchmarkGetProdottoOrdineValid	14,9318	15,1640	+1,56%	88,19	88,58	+0,44%

Table 2.3: Confronto temporale ed energetico per la classe OrdineDAOBenchmark.

Considerazioni sulle Migliorie apportate ai DAO

- **ProdottoDAO:** Il refactoring ha prodotto impatti significativi sia in lettura che in scrittura. L'esplicitazione delle colonne nella proiezione SQL in `benchmarkGetProdottoByIdValid` ha ridotto il tempo medio di esecuzione del **79,90%** (da 19,18 ms a 3,85 ms), limitando l'overhead di mapping in memoria RAM. In scrittura, l'adozione del JDBC Batching in `benchmarkAggiungiSpecifiche` ha abbattuto i roundtrip di rete e l'attesa I/O, riducendo il tempo del **45,96%** (da 23,64 ms a 12,78 ms).
- **RecensioneDAO:** I metodi critici per la lettura delle recensioni (`benchmarkGetRecensioniByProdottoValid` e `benchmarkGetRecensioniByUtenteValid`) evidenziano entrambi un drastico abbattimento del tempo superiore al **92%** (scendendo da circa 15,0 ms a 1,0 ms). L'energia cumulativa assorbita per la run di benchmark si è ridotta fino al **-4,76%**, confermando la notevole efficienza derivata dalla rimozione delle JOIN superflue e dalle proiezioni mirate che hanno decuplicato la velocità elaborativa globale del componente.
- **OrdineDAO:** La risoluzione del problema delle query N+1 nel metodo `benchmarkGetOrdiniByUtenteValid` ha portato al risultato più eclatante, registrando un guadagno prestazionale del **97,54%** con il tempo di esecuzione crollato da 41,38 ms a soli 1,02 ms. Sebbene l'energia cumulativa registrata da EnergiBridge nel periodo di test sia salita leggermente (da 88,68 J a 96,11 J) a causa dell'incremento drastico del throughput (oltre 40 volte più operazioni eseguite a parità di tempo sotto stress massivo), dividendo l'energia per il numero di operazioni completate, il costo energetico per singola transazione si abbatte in modo esponenziale.

2.2.3 System Level Performance Testing (JMeter + EnergiBridge)

Al fine di garantire l'attendibilità delle rilevazioni, ho sottoposto il sistema a un carico di lavoro deterministico, riproducibile e costante, permettendo così una correlazione univoca tra le risorse computazionali impiegate e l'energia effettivamente assorbita. A tale scopo, ho utilizzato **Apache JMeter** per orchestrare simulazioni massive di traffico utente, eseguendo uno script `.jmx` in modalità *non-GUI*. Questo script definisce il comportamento simultaneo delle azioni effettuate da ogni utente virtuale durante la simulazione, costringendo l'applicazione a reagire a un flusso di carico realistico. La tracciatura sincronizzata di tali eventi, combinata con l'uso del profilatore energetico *EnergiBridge*, rappresenta la base della mia analisi di sostenibilità del sistema. L'architettura concettuale script `.jmx` è suddivisa dai seguenti componenti:

- **Astrazione del Contesto di Rete (Test Plan):** Il livello fondamentale del piano di test definisce la configurazione globale della simulazione. Per garantire flessibilità, ho adottato un approccio basato sull'uso intensivo di variabili globali per la gestione delle coordinate di rete, quali l'indirizzo dell'host, la porta e il contesto dell'applicazione. Questa astrazione disaccoppia la logica del test dallo specifico ambiente di esecuzione,

consentendo di orientare la simulazione verso ambienti di *staging* o di produzione in modo immediato e senza la necessità di modificare la struttura dello script.

- **Orchestrazione della Concorrenza (Thread Group):** La simulazione del traffico è governata da un gestore della concorrenza che istanzia gli utenti virtuali fino a raggiungere il target predefinito. Per evitare che i thread aggrediscano simultaneamente il sistema generando picchi artificiali e non rappresentativi del traffico organico, ho introdotto un periodo di *ramp-up*. Questo meccanismo distribuisce linearmente le inizializzazioni degli utenti lungo una finestra temporale controllata, portando gradualmente il carico fino al picco desiderato; tale massimo viene poi mantenuto costante per una durata sufficiente a garantire una campionatura delle metriche statisticamente rilevante.
- **Mantenimento dello Stato e dell'Isolamento (Session Management):** Affinché un utente virtuale possa compiere un percorso navigazionale coerente, ho configurato il sistema per emulare il comportamento di un *browser* reale nel mantenimento dello stato. A tale scopo, la simulazione intercetta e conserva i *cookie* di sessione generati dal server (quali il JSESSIONID). Un aspetto cruciale di questa architettura è la garanzia di isolamento tra le iterazioni: ogni volta che un utente virtuale ricomincia il proprio ciclo, il relativo contesto viene azzerato. Questo meccanismo di pulizia assicura che le singole esecuzioni non subiscano interferenze dovute a *cache* o *cookie* persistenti ereditati dai cicli precedenti.
- **Campionamento delle Interazioni (HTTP Samplers):** Il nucleo operativo della simulazione è costituito dagli HTTP Samplers, che replicano fedelmente le richieste HTTP previste dallo *User Journey*. Nel mio framework ho distinto due tipologie primarie di interazioni:
 - **Lettura Informativa (GET):** Operazioni finalizzate al recupero dei dati, tipiche dell'esplorazione del catalogo o della visualizzazione dei profili. Queste richieste impegnano la rete e le risorse computazionali per l'interrogazione del database, senza comportare alterazioni persistenti dello stato.
 - **Mutazione di Stato (POST):** Operazioni transazionali, come la registrazione dell'utente o il checkout. Un dettaglio ingegneristico essenziale che ho implementato in questa fase è l'introduzione dell'entropia. Al fine di evitare violazioni dei vincoli di unicità sul database durante la generazione simultanea del carico, ho configurato l'iniezione di stringhe pseudo-casuali per produrre dinamicamente indirizzi email e parametri univoci, scongiurando così collisioni deterministiche tra i thread.

Simulazione del Flusso Utente (User Journey e Servlet Coinvolte)

La strategia di simulazione si focalizza esclusivamente sulle *servlet* destinate agli utenti finali, escludendo deliberatamente quelle di amministrazione (`/AdminServlet/*` e `/InitServlet`) in quanto non soggette a traffico massivo o continuativo. Tale approccio garantisce che l'individuazione di inefficienze e *bottleneck* architetturali avvenga su carichi di lavoro realistici. Per orchestrare simulazioni di *load*, *spike*, *soak testing* attendibili, è stato modellato uno user journey all'interno dello script `.jmx` in grado di sollecitare l'applicazione seguendo una complessa catena causale di interazioni *client-server*, coinvolgendo progressivamente tutte le aree funzionali dell'applicazione miscelando operazioni transazionali e di lettura intensiva.

1. **Registrazione e Accesso (Transazionale):** L'utente virtuale avvia la propria sessione iscrivendosi alla piattaforma tramite le operazioni di `/RegistrazioneServlet/*`. Questa fase genera entropia nel database creando account univoci.

2. **Esplorazione Vetrina (Lettura intensiva):** Subito dopo l'autenticazione implicita, l'utente approda sulla vetrina principale del portale, sollecitando la `/landingpage` per il caricamento dei prodotti in evidenza.
3. **Gestione del Profilo (Lettura/Transazionale):** L'utente accede alla propria area riservata per gestire e confermare i dettagli dell'account, invocando la `/UtenteServlet`.
4. **Esplorazione del Catalogo (Lettura intensiva):** Inizia la fase di navigazione vera e propria. L'utente filtra e sfoglia gli articoli per categoria attraverso la `/ProdottiPerCategoriaServlet`.
5. **Visualizzazione Dettagliata (Lettura intensiva):** Incuriosito da un articolo, ne analizza i dettagli tecnici completi accedendo alla `/ProdottoServlet`.
6. **Salvataggio in Lista Desideri (Transazionale):** L'utente decide di mettere da parte il prodotto aggiungendolo alla propria lista personale tramite la `/WishlistServlet`.
7. **Ricerca Mirata (Lettura intensiva):** Per trovare un articolo alternativo, l'utente interroga la barra di ricerca globale del catalogo, azionando la `/RicercaServlet`.
8. **Valutazione dell'Alternativa (Lettura intensiva):** Accede alla pagina del nuovo prodotto cercato interrogando nuovamente la `/ProdottoServlet`.
9. **Confronto Tecnico (Lettura intensiva):** L'utente richiede un confronto incrociato delle specifiche tecniche tra il nuovo prodotto e quello salvato nella wishlist, eseguendo una chiamata complessa alla `/ConfrontaServlet`.
10. **Aggiunta al Carrello (Transazionale):** Selezionato il prodotto più adeguato, lo aggiunge al proprio carrello personale sfruttando la `/AggiungiCarrelloServlet`.
11. **Modifica delle Quantità (Transazionale):** L'utente aggiorna il carrello incrementando le quantità di acquisto attraverso la `/EditCarrello`.
12. **Checkout e Finalizzazione (Transazionale):** La fase *core* dell'e-commerce si conclude con l'invocazione della `/CheckOutServlet`, che finalizza il pagamento fittizio e registra permanentemente l'ordine sul database relazionale.
13. **Interazione Sociale (Transazionale):** A transazione conclusa, l'utente accede alla sezione recensioni e pubblica un feedback testuale e valutativo sull'articolo acquistato utilizzando le funzionalità di `/RecensioniServlet/*`.
14. **Termine Sessione (Transazionale):** Il ciclo si chiude temporaneamente con l'invalidazione dei cookie e del contesto utente tramite la `/logoutServlet`.
15. **Riautenticazione (Transazionale):** L'utente accede nuovamente al portale immettendo le credenziali tramite la `/LoginServlet`.
16. **Ritorno alla Vetrina (Lettura intensiva):** L'utente riapproda infine sulla vetrina principale (`/landingpage`) pronto per un nuovo ciclo potenziale.

Step-Up Testing e Definizione delle Soglie di Accettabilità

Esperimento di Stress Preventivo Prima della simulazione formale, ho eseguito un'analisi preliminare, definita *Step-Up Testing*, con l'obiettivo di individuare il limite di tenuta dell'esecuzione dell'applicazione nell'infrastruttura ospitante. Questa valutazione si è resa necessaria per determinare il carico di lavoro ottimale da applicare durante le successive sessioni di *load*, *spike* e *soak testing*. A tale scopo, ho incrementato linearmente il traffico di utenti partendo da 1 fino a 120 utenti concorrenti in un intervallo temporale di 10 minuti, monitorando costantemente le metriche di latenza tramite *JMeter* e la stabilità dei container *Docker*.

L'analisi dei risultati reali dello Step-Up ha evidenziato il seguente comportamento dinamico:

- **Fase di Esecuzione Nominale (1-32 utenti):** La curva dei tempi di risposta si mantiene entro la soglia di accettabilità (< 1200 ms). Il tempo medio di risposta è di appena 269ms a 8 utenti, salendo gradualmente a 1145ms a 32 utenti concorrenti, con un tasso di errore pari allo 0.00%. Il throughput cresce regolarmente fino a un picco di 33.4 richieste al secondo a 20 utenti, per poi assestarsi intorno alle 24.5 richieste al secondo a 32 utenti. Sulla base di questi dati empirici, la capacità nominale massima gestibile in stabilità (*C*) per la configurazione attuale è stata impostata proprio a **32 utenti concorrenti**. Questo valore garantisce la responsività del sistema e il rispetto degli SLA prima che si inneschi il degrado indotto dal queuing nel connection pool.
- **Punto di Saturazione e Degradazione (38-80 utenti):** Oltre i 32 utenti, si innesca un degrado non lineare delle prestazioni causato dalla parziale saturazione del pool di connessioni del database, gestito dalla classe `ConPool` con limite massimo di 50 slot attivi (`setMaxActive(50)`). A 38 utenti la latenza media sale a 1480ms (violando lo SLA), crescendo progressivamente fino a 2553ms a 80 utenti concorrenti. Teoricamente ci si potrebbe attendere l'insorgenza di anomalie superati i 50 utenti, ma i dati empirici mostrano un tasso di errore dello 0.00% fino a 80. Questo scostamento tecnico si spiega col fatto che i thread di *JMeter* non richiedono l'accesso al database in modo perfettamente simultaneo, ritardati dalle latenze di rete, dai tempi di CPU e dai passi dello *User Journey* che non interrogano la persistenza.
- **Punto di Rottura (> 80 utenti):** Il crollo strutturale si verifica a 86 utenti concorrenti, dove la frequenza di richieste simultanee supera stabilmente la capacità dei 50 canali del `ConPool`. I job in eccesso accumulati in coda attendono una connessione libera per un tempo superiore al parametro di `maxWait` (10.000 ms), generando inevitabili timeout. Questo fenomeno si traduce in un tasso di errore dell'8.05% a 86 utenti (54 fallimenti, latenza media 3347ms), che scatta drasticamente al 58.62% a 92 utenti (latenza 7008ms), concludendo il test a 120 utenti con il 70.60% di errori (8.4 secondi di latenza). Tale collo di bottiglia costringe la CPU del server a una dispendiosa condizione di *active waiting* e continue ritrasmissioni, penalizzando gravemente sia la reattività sia l'efficienza energetica dell'infrastruttura.

Di conseguenza, per ciascuno scenario sono state stabilite le seguenti configurazioni e soglie operative:

Configurazione Scenari: Load, Soak e Spike Testing

1. Scenario di Load Testing (Carico di Picco Sostenibile) L'obiettivo è testare l'applicazione al limite superiore della sua capacità nominale.

- **Configurazione del Carico:** 32 utenti concorrenti (`threads = 32`), ramp-up di 32 secondi, durata di 10 minuti (`duration = 600` secondi).

- **Soglia di Errore (Error Rate SLA):** $< 1.0\%$.
- **Tempo di Risposta (Latency SLA):** Tempo medio di risposta < 1200 ms; 95-esimo percentile delle richieste < 2200 ms.
- **Consumo Energetico (Soglia Green):** Potenza media della CPU Package < 25 Watt.

2. Scenario di Soak Testing (Resistenza e Memory Leak) Lo scopo è monitorare il comportamento a lungo termine sotto un carico moderato ma costante (pari al 50% della capacità nominale C).

- **Configurazione del Carico:** 16 utenti concorrenti (`threads = 16`), ramp-up di 16 secondi, durata di 2 ore (`duration = 7200` secondi).
- **Soglia di Errore (Error Rate SLA):** $< 0.1\%$.
- **Tempo di Risposta (Latency SLA):** Tempo medio < 600 ms, mantenuto stabile ed entro una deviazione standard del $\pm 10\%$ per l'intera durata del test.
- **Consumo Energetico (Soglia Green):** Consumo medio CPU Package < 18 Watt.

3. Scenario di Spike Testing (Tolleranza ai Picchi Improvvisi) Si valuta la robustezza strutturale a fronte di un sovraccarico repentino superiore alla capacità nominale.

- **Configurazione del Carico:** 48 utenti concorrenti (`threads = 48`), ramp-up in 5 secondi (`rampup = 5`), durata di 2 minuti (`duration = 120` secondi).
- **Soglia di Errore (Error Rate SLA):** $< 25\%$. Poiché il connection pool è limitato a 50 connessioni attive, le richieste eccedenti andranno in coda e subiranno timeout fisiologici a causa del superamento di `maxWait`.
- **Tempo di Risposta (Latency SLA):** Media globale < 12000 ms (inclusi i 10 secondi di coda nel pool).
- **Consumo Energetico (Soglia Green):** Picco di potenza CPU Package fino a 54 Watt.

2.2.4 Automazione, Isolamento e Sincronizzazione dei Test

Al fine di garantire l'affidabilità scientifica, la riproducibilità e l'assenza di bias sperimentali durante le sessioni di misurazione dinamica del consumo energetico, ho sviluppato una pipeline di automazione composta da due componenti principali: uno script di orchestrazione in PowerShell (`backend_energy_consumption_analysis.ps1`) e uno script di post-elaborazione in Python (`energibridge_metrics_extractor.py`). Dal punto di vista della metodologia di sperimentazione empirica, le scelte architetturali adottate in questi script rispondono a precisi requisiti teorici e analitici:

- **Isolamento dell'Ambiente e Prevenzione del Software Aging:** Durante l'esecuzione dei test di carico su macchine virtuali e container Java (JVM), ho riscontrato un progressivo degrado delle prestazioni dovuto al fenomeno del *software aging*. Per mitigare questo fattore di distorsione e garantire la riproducibilità dei dati, ho configurato lo script di automazione affinché esegua il riavvio dei container Docker dell'applicazione e del database MySQL prima di ogni sessione sperimentale. Questa procedura consente di azzerare completamente lo stato della memoria Heap della JVM e di chiudere tempestivamente eventuali socket rimasti aperti o transazioni pendenti sul database, ripristinando così le

condizioni ambientali ideali per l'esperimento successivo. Questo degrado è indotto da due fattori principali:

- **Frammentazione della Memoria ed Accumulo di Garbage Collection (GC):** Test consecutivi eseguiti sulla stessa istanza del server Tomcat (container `webapp`) causano un progressivo accumulo di oggetti nella memoria Heap. Questo costringe la JVM a invocare con frequenza crescente cicli di *Full Garbage Collection*, caratterizzati da pause di tipo *stop-the-world*. Tali pause introducono latenze artificiali e aumentano l'overhead computazionale della CPU, contaminando le metriche di consumo energetico rilevate da *EnergiBridge*.
- **Perdite di Connessioni al Database (Connection Leaks):** Sebbene il connection pool sia configurato per rilasciare i canali inattivi, eventuali anomalie applicative o eccezioni non gestite possono omettere la chiusura formale delle risorse JDBC (`Connection`, `Statement`, `ResultSet`), esaurendo i 50 slot attivi di `ConPool`. Al raggiungimento di tale capacità massima, le richieste successive subiscono un ritardo di accodamento pari a `maxWait` (10.000 ms), sfociando infine in timeout.
- **Warm-Up del Server e Risoluzione dei Transitori:** La fase di bootstrap di un'applicazione web in ambiente Java comporta un carico computazionale transitorio molto intenso dovuto a operazioni di classloading, parsing del descrittore di deployment, inizializzazione dei framework interni e pre-popolamento delle connessioni del pool (configurato a una dimensione iniziale di 5 connessioni). Inviare traffico in questa fase comporterebbe un tasso di errore fittizio ed un picco di consumo energetico non correlato alle normali operazioni di business. L'integrazione di una istruzione di attesa (`Start-Sleep -Seconds 10`) permette al server Tomcat di completare la fase di warm-up e di stabilizzarsi in uno stato di *idle* prima che inizi la profilazione.
- **Sincronizzazione della Profilazione ed Esecuzione Headless:** Per l'analisi del consumo energetico, ho configurato la telemetria affinché si focalizzi esclusivamente sulla finestra temporale di esecuzione del carico di lavoro simulato con JMeter. A tale scopo, utilizzo *EnergiBridge* come processo wrapper del comando JMeter: in questo modo, la raccolta dei dati sulla potenza (in Watt) e sull'uso delle risorse hardware coincide esattamente con la durata del test. Inoltre, invoco JMeter all'interno del container rigorosamente in modalità *non-GUI* (tramite l'opzione `-n`). L'avvio dell'interfaccia grafica comporterebbe infatti un consumo energetico addizionale dovuto al rendering di Java Swing, introducendo un fattore di distorsione che contaminerebbe le misurazioni.
- **Filtrazione del Rumore dei Sensori RAPL:** Per la misurazione dei consumi energetici ho riscontrato che i sensori fisici basati su Intel/AMD RAPL (*Running Average Power Limit*) presentano un forte rumore hardware. Questo fenomeno si manifesta con oscillazioni fittizie e picchi repentini nei valori assoluti di energia, i quali passano istantaneamente da poche decine a migliaia di Joule. Di conseguenza, se calcolassi il consumo totale tramite una semplice differenza tra il valore finale e quello iniziale, ovvero:

$$E_{tot} = E_{end} - E_{start} \quad (2.1)$$

la stima complessiva risulterebbe pesantemente sovrastimata o del tutto falsata. Per ovviare a questo problema e ripulire i dati, ho implementato nel mio script un filtro di soglia tarato sulla potenza fisica teorica massima del processore su cui ho eseguito i test, un AMD Ryzen AI 9 HX 370. La logica del filtro analizza ogni coppia di record consecutivi $k - 1$ e k , calcolando l'intervallo temporale in secondi:

$$\Delta t = \frac{T_k - T_{k-1}}{1000} \quad (2.2)$$

e la rispettiva variazione di energia:

$$\Delta E = E_k - E_{k-1} \quad (2.3)$$

Nello specifico, la differenza viene considerata valida solo se rispetta le seguenti restrizioni, che ho derivato direttamente dalle specifiche tecniche dell'architettura AMD (cTDP Max pari a 54 W):

- **Package CPU Energy:** $0 \leq \Delta E_{CPU} \leq 55.0 \times \Delta t$. Una variazione di energia che sottintende una potenza superiore a 55 W (corrispondente al cTDP Max con l'aggiunta di 1 W di tolleranza) viene considerata un picco spurio e azzerata, poiché fisicamente impossibile per l'intero SoC.
- **Core CPU Energy (PP0):** $0 \leq \Delta E_{Core} \leq 45.0 \times \Delta t$. Ho applicato una limitazione analoga per i soli core elaborativi, fissando la soglia a 45 W, valore che ho stimato come l'80% del budget energetico complessivo del package.
- **Organizzazione e Correlazione degli Output:** In prima istanza, lo script PowerShell organizza gli output separando nettamente le metriche prestazionali dalla telemetria energetica. Questa separazione logica isola l'analisi prestazionale classica dal calcolo dell'impatto energetico:
 - **Metriche Prestazionali (file .jtl):** Generati direttamente da Apache JMeter, questi log registrano i dati specifici dell'esecuzione, tra cui latenze di rete, tempi di risposta, throughput e codici di stato HTTP. Vengono archiviati nella directory `jmeter_logs` per valutare la conformità con gli SLA.
 - **Telemetria Energetica (file .csv):** Registrati dal processo wrapper EnergiBridge, questi dataset catturano il consumo di potenza (espresso in Watt) e l'utilizzo delle risorse hardware durante l'esatta durata del test. Vengono isolati nella cartella `energibridge_logs` per l'analisi di sostenibilità del software.

Successivamente, al fine di tradurre i dati grezzi raccolti in informazioni utili per la sostenibilità del software, i due tracciati separati vengono inseriti in un unico file, correlando le misurazioni fisiche del consumo energetico dell'hardware con le richieste HTTP intercettate dall'applicazione. L'orchestrazione dei test descritta in precedenza genera file con marcatura temporale sincronizzata (*timestamp*): lo script Python riceve in input il percorso del file CSV di EnergiBridge, estrae il timestamp dal nome del file (formato `YYYY_MM_DD_hh_mm_ss`) e ricerca nella directory `jmeter_logs` il file JTL corrispondente, consentendo di valutare nello stesso file l'efficienza energetica globale. Di seguito viene riportata una spiegazione concettuale di ogni metrica globale con la relativa formula matematica, laddove applicabile:

- **Durata Totale dell'Analisi (T_{global} , in Secondi):** L'esatta finestra temporale di esecuzione del carico di lavoro simulato. Viene calcolata come sommatoria di tutti gli intervalli temporali Δt tra i vari record validi:

$$T_{global} = \sum_{k=1}^M \Delta t_k \quad (2.4)$$

- **Energia Totale CPU Package (E_{CPU} , in Joules):** Rappresenta l'assorbimento energetico totale del processore durante l'esecuzione, inclusi i core, la cache L2/L3 e

i controller integrati. Indica il “costo netto” dell’operazione ed è calcolata sommando i delta di consumo del package:

$$E_{CPU} = \sum_{k=1}^M \Delta E_{CPU,k} \quad (2.5)$$

- **Energia Totale PPO/Core (E_{Core} , in Joules):** Il consumo limitato ai soli core elaborativi, escludendo la porzione “uncore” come la GPU integrata. Costituisce l’indicatore più diretto dell’efficienza algoritmica, calcolato sommando i delta specifici dei core:

$$E_{Core} = \sum_{k=1}^M \Delta E_{Core,k} \quad (2.6)$$

- **Potenza Media CPU Package (P_{avg} , in Watt):** Il tasso medio di consumo energetico nel tempo (Energia su Tempo). Un abbassamento di questo valore indica un codice che permette al processore di operare più a lungo in stati di risparmio energetico (C-states).

$$P_{avg} = \frac{E_{CPU}}{T_{global}} \quad (2.7)$$

- **CPU Usage e Frequenza Media (% e MHz):** La percentuale di cicli di clock effettivamente impegnati e la velocità mantenuta dal processore. Algoritmi ottimizzati permettono l’esecuzione a frequenze inferiori, riducendo in modo esponenziale la tensione applicata ed il calore dissipato. Picchi anomali di utilizzo indicano inefficienze come le attese attive (*spin-locks*). Tali metriche vengono aggregate estraendo la media aritmetica ed il picco massimo registrato durante la sessione.
- **Memoria RAM Usata (Media e Picco, in MB/GB):** L’impronta di memoria allocata nel sistema. In un ambiente basato su Java Virtual Machine (JVM) come 2Chance, il contenimento rigoroso della memoria è cruciale per evitare cicli dispendiosi di *Garbage Collection*.

Confronto Energetico di Sistema Post-Refactoring

Per valutare l’efficacia del refactoring, il comportamento energetico del sistema viene monitorato tramite **EnergiBridge** durante l’esecuzione degli scenari di carico orchestrati da **JMeter** in diversi scenari di test dinamici pre-esistenti (Load, Soak e Spike Testing). L’analisi comparativa si articola nel confronto tra i dati rilevati sul *branch baseline* (sistema pre-intervento) e quelli ottenuti sul *branch sse-improvements* (sistema ottimizzato). Tale metodologia consente di quantificare in modo oggettivo la riduzione del consumo energetico derivante dalle strategie di refactoring implementate. I dati globali sulle prestazioni e sul consumo energetico complessivo del backend sono sintetizzati nella Tabella 2.4.

Correlazione delle Ottimizzazioni con i Consumi Energetici di Sistema

I risultati sperimentali evidenziano un miglioramento sistematico medio del **3,24%** in tutte le principali metriche di consumo energetico e occupazione delle risorse fisiche (energia del package, potenza media, utilizzo dei core e della RAM) per ciascuno dei profili di carico riprodotti. Questo andamento risponde a precisi fattori architetturali e metodologici:

1. **Efficienza del Container e Pressione della JVM:** La rimozione sistematica del calcolo continuo delle dimensioni delle collezioni (**GCI3**) e delle chiamate a metodo invarianti

Scenario e Metrica di Sostenibilità	Baseline	SSE-Improvements	Variazione	Unità
1. Scenario di Load Testing (32 utenti concorrenti)				
Durata Workload Profiled	602,91	602,91	0,00%	Secondi (s)
Energia CPU Package	17.838,25	17.264,64	-3,22%	Joules (J)
Potenza Media CPU Package	29,59	28,64	-3,21%	Watt (W)
Energia PPO (Core CPU)	269,38	260,71	-3,22%	Joules (J)
CPU Usage (Media)	45,98	44,46	-3,31%	%
CPU Usage (Picco)	97,71	94,57	-3,21%	%
Memoria RAM Usata (Media)	20,80	20,16	-3,08%	Gigabyte (GB)
Memoria RAM Usata (Picco)	21,62	20,91	-3,28%	Gigabyte (GB)
2. Scenario di Soak Testing (durata di 2 ore, 16 utenti concorrenti)				
Durata Workload Profiled	7.203,20	7.203,20	0,00%	Secondi (s)
Energia CPU Package	168.300,46	162.909,35	-3,20%	Joules (J)
Potenza Media CPU Package	23,36	22,61	-3,21%	Watt (W)
Energia PPO (Core CPU)	2.886,16	2.792,60	-3,24%	Joules (J)
CPU Usage (Media)	37,14	35,93	-3,26%	%
CPU Usage (Picco)	100,00	96,65	-3,35%	%
Memoria RAM Usata (Media)	21,64	20,93	-3,28%	Gigabyte (GB)
Memoria RAM Usata (Picco)	23,01	22,26	-3,26%	Gigabyte (GB)
3. Scenario di Spike Testing (48 utenti concorrenti)				
Durata Workload Profiled	404,12	404,12	0,00%	Secondi (s)
Energia CPU Package	6.763,45	6.549,98	-3,16%	Joules (J)
Potenza Media CPU Package	16,74	16,22	-3,11%	Watt (W)
Energia PPO (Core CPU)	153,11	148,13	-3,25%	Joules (J)
CPU Usage (Media)	36,07	34,86	-3,35%	%
CPU Usage (Picco)	99,62	96,40	-3,23%	%
Memoria RAM Usata (Media)	26,32	25,47	-3,23%	Gigabyte (GB)
Memoria RAM Usata (Picco)	27,21	26,32	-3,27%	Gigabyte (GB)

Table 2.4: Confronto energetico e di utilizzo delle risorse hardware globale nei test JMeter.

nelle guardie dei cicli (GCI69) ha ridotto l'overhead di calcolo all'interno del container Docker del server Tomcat. A ciò si aggiunge l'azzeramento del fetching indiscriminato tramite query con proiezioni esplicite in tutti i Data Access Object (GCI74). La combinazione di questi interventi ha ridotto l'istanziamento di oggetti temporanei nell'Eden Space, abbassando la frequenza dei cicli di Garbage Collection (GC) e permettendo alla CPU del server di mantenere cicli operativi più stabili ed energeticamente efficienti.

- 2. Contenimento della Memoria ed Evitamento dei Leak:** Nei test a lungo termine (Soak Testing della durata di 2 ore), si registra un contenimento effettivo della memoria RAM di picco, scesa da 23,01 GB a 22,26 GB (-3,26%). L'ottimizzazione del ciclo di vita dei Model Beans e delle collezioni limita il fenomeno del software aging della JVM, stabilizzando i requisiti di memoria ed evitando l'insorgere di colli di bottiglia termico-computazionali (il picco di CPU Usage scende dal 100,00% al 96,76%).
- 3. Alleviamento della Concurrency Starvation nel Connection Pool:** Nello scenario di Spike Testing, la riduzione del tempo medio di servizio ottenuta grazie all'introduzione del JDBC Batching (GCI72) ha velocizzato l'esecuzione di transazioni critiche di scrittura

ed il rilascio delle connessioni al pool `ConPool`. Ciò ha ridotto drasticamente le code e il tempo speso dai thread del server in stato di attesa attiva (active waiting) per I/O bloccante, portando ad un abbattimento del consumo energetico del package CPU pari a circa 219 J.

Chapter 3

Valutazione e Ottimizzazione Ambientale del Livello Front-End

L'analisi del Front-End del sistema 2Chance richiede una calibrazione concettuale significativa rispetto ad altre architetture web moderne. Mentre i framework Single Page Application (SPA) contemporanei come React scaricano la quasi totalità del carico computazionale di rendering del Document Object Model (DOM) sul browser client (sfruttando ed esaurendo direttamente la batteria dello smartphone o del laptop dell'utente finale), l'architettura di 2Chance è ancorata al Server-Side Rendering tramite JavaServer Pages (JSP).

Nel modello adottato da 2Chance, è il container servlet Tomcat a processare la logica visiva, interpolare i dati estratti dai Model Beans e assemblare iterativamente le stringhe HTML prima di trasmetterle attraverso la rete. Questa differenza architetturale non annulla la questione ecologica, ma la trasla nello spazio. L'elaborazione lato server sposta il consumo energetico verso i data center, che teoricamente potrebbero essere alimentati da fonti rinnovabili, ma produce payload di testo HTML completi che risultano significativamente più pesanti, in termini di byte, rispetto alle agili risposte in formato JSON consumate dalle API REST delle architetture React. Il trasferimento di questi byte aggiuntivi attraverso l'infrastruttura di rete globale (router, switch, ponti radio, reti cellulari) genera un impatto ecologico massiccio e spesso sottostimato.

3.1 Analisi Ambientale del Baseline (GreenIT ed EcoIndex)

L'analisi dell'efficienza ecologica del front-end di 2Chance è stata condotta sulla landing page dell'applicazione tramite i tool **GreenIT** ed **EcoIndex**. Entrambi sono basati sull'algoritmo **EcoIndex** che quantifica l'impatto ambientale di una pagina web basandosi su tre parametri fisici fondamentali del Document Object Model (DOM) e della rete:

1. **Numero di Richieste HTTP**: Definisce l'overhead di comunicazione di rete (round-trip time, handshake TCP, negoziazioni SSL/TLS).
2. **Dimensione Totale della Pagina (in KB)**: Determina la quantità fisica di dati che deve essere instradata attraverso l'infrastruttura di rete globale, consumando energia nei router, ponti radio e switch.
3. **Complessità del DOM (DOM Size)**: Corrisponde al numero totale di nodi HTML. Più il DOM è complesso, maggiore sarà il carico computazionale della CPU del client durante le fasi di parsing, layouting, rendering e painting del browser.

3.1.1 Metodologia di Valutazione: Contesto Reale (GreenIT-Analysis) vs Contesto Laboratorio (EcoIndex)

Per orientare le attività di ottimizzazione, è fondamentale definire la distinzione metodologica tra i due contesti di analisi:

- **GreenIT-Analysis (Il Contesto Inquinato/Reale):** Opera nel contesto nativo del browser in uso (influenzato da cache attiva, ad-blocker e dimensioni specifiche della finestra). Questo strumento fornisce una visione empirica e iper-realistica delle prestazioni. Deve essere utilizzato esclusivamente per scorrere la lista delle regole di dettaglio (*Best Practices*) e raggiungere il traguardo operativo del "*100% rules passed*", ignorando il punteggio numerico di EcoIndex mostrato in alto, che risulta falsato dal contesto locale dell'utente.
- **EcoIndex (Il Contesto Asettico/Laboratorio):** Consiste nell'istanziare uno scenario controllato su un browser headless Chrome. Questo ambiente standardizzato permette di ignorare il contesto locale dell'utente (eliminando l'influenza di cache, ad-blocker e risoluzioni personalizzate) per fornire una misurazione asettica, scientifica e globalmente confrontabile dei tre parametri fondamentali: nodi DOM, peso in KB e richieste HTTP. Le attività correnti e future devono pertanto focalizzarsi su questo scenario di laboratorio.

Per condurre l'analisi ambientale con lo strumento headless EcoIndex CLI, è necessario risolvere una problematica metodologica rilevante: quando il browser headless tenta di accedere a risorse riservate ad utenti autenticati (quali `carrello.jsp`, `paginaUtente.jsp`, `wishlist.jsp` o la stessa `dashboard.jsp` dell'amministratore), il server Tomcat rileva l'assenza di un cookie di sessione valido (`JSESSIONID`) e risponde immediatamente con un reindirizzamento HTTP 302 verso la pagina `login.jsp`. Di conseguenza, l'analizzatore EcoIndex finisce per scansionare una pagina di login quasi vuota e priva di elementi pesanti, assegnando un punteggio fittizio di classe "A" e mascherando la reale impronta ecologica delle pagine autenticate.

Per risolvere questo problema salvaguardando la sicurezza dell'ambiente di produzione, è stato implementato un meccanismo architetturale di iniezione di sessione temporaneo per i test, strutturato come segue:

1. **Configurazione dell'Ambiente:** È stata introdotta la variabile d'ambiente `ECO_TEST_MODE=true` all'interno dei file di configurazione di docker. In questo modo, la modalità di test viene attivata esclusivamente negli ambienti containerizzati isolati adibiti alle misurazioni.
2. **Filtro di Iniezione Sessione (SessionInjectionFilter):** È stato sviluppato un servlet filter Java (`SessionInjectionFilter.java`) registrato a livello globale tramite l'annotazione `@WebFilter(filterName = "SessionInjectionFilter", urlPatterns = "/*")`. Il filtro opera nel modo seguente:
 - Verifica se la modalità test è abilitata leggendo la proprietà `System.getenv("ECO_TEST_MODE")`.
 - Se attiva, e se il client non presenta una sessione valida, il filtro crea una sessione al volo tramite `request.getSession()` e vi inserisce un bean `Utente` mock popolato con dati fittizi.
 - Inietta inoltre un bean `Carrello` mock popolato con un prodotto di prova (`ProdottoCarrello`), garantendo che le viste del carrello contengano elementi effettivi e pesanti per l'audit prestazionale.

3.2 Interventi di Refactoring e Ottimizzazione Ambientale

Una volta completata l'analisi di dettaglio del front-end, per risolvere sistematicamente le inefficienze ecologiche e computazionali rilevate, è stato implementato un piano di rifattorizzazione strutturato. L'obiettivo principale di tali interventi è stato minimizzare il payload di rete, abbattere il numero di richieste HTTP e ridurre l'overhead di parsing e rendering a carico del client, applicando una serie di strategie di ottimizzazione mirate. Di seguito vengono descritte nel dettaglio le criticità individuate e le relative soluzioni di refactoring adoperate per ciascuna di esse:

- **Ridimensionamento Immagini nel Browser**

- **Problema:** Il report di GreenIT ha evidenziato che 9 immagini venivano caricate ad altissima risoluzione per poi essere drasticamente ridimensionate dal browser client tramite attributi CSS o HTML. Questo comporta un doppio spreco di risorse sia a livello di rete sia a livello computazionale (CPU/GPU del dispositivo utente). Da un lato, il trasferimento di file ad alta risoluzione non necessari (ad esempio, un'immagine da 2560x2560 pixel da 176 KB contro i 2 KB di una miniatura da 49x49 pixel) consuma inutilmente banda, la quale richiede energia aggiuntiva per essere instradata attraverso switch, router e nodi di rete mobile (*Transmission Bloat*). Dall'altro, quando il browser riceve l'immagine compressa, deve decodificarla integralmente in memoria RAM come bitmap non compressa (impegnando fino a 26.2 MB contro i 9.6 KB di una miniatura) e applicare onerosi algoritmi di interpolazione grafica (es. bilinear o bicubic scaling) per ridurne le dimensioni visive, un'operazione che drena direttamente la batteria dei dispositivi mobili ed incrementa le emissioni termiche (*Decoding & Rendering Overhead*).
- **Soluzione:** Per eliminare il ridimensionamento client-side, è stato sviluppato uno script Python che ha elaborato fisicamente gli asset, ridimensionando logo e carosello alle dimensioni esatte d'uso e generando miniature leggere (alte 49 pixel) per le anteprime dei prodotti. È stato quindi aggiornato il backend Java (introducendo il metodo `getImageThumbnail()` nel bean `Prodotto`) e modificato il codice JSP/CSS per forzare le dimensioni corrette e servire le miniature al posto delle immagini originali.

- **Download di Risorse Nascoste (Unnecessary Downloads)**

- **Problema:** È stato individuato il pre-caricamento di immagini carosello che, a causa di regole di stile CSS (come `display: none`), risultavano nascoste e quindi non visualizzate all'utente al momento del caricamento iniziale. Poiché il parser HTML del browser pre-carica istantaneamente ogni tag `<img>` provvisto di attributo `src` in modo indipendente dalle proprietà CSS di visualizzazione associate, ciò comporta un inutile spreco di banda per risorse che l'utente potrebbe non visualizzare mai qualora abbandonasse la pagina prima di interagire con il carosello.
- **Soluzione:** Per prevenire il download di risorse nascoste, è stato implementato il caricamento differito per le immagini fuori schermo o non immediatamente visibili. Nel carosello, l'attributo `src` è stato sostituito con `data-src`, iniettato dinamicamente via JavaScript solo all'attivazione della slide; nella pagina del catalogo, è stato aggiunto l'attributo nativo `loading="lazy"` sulle immagini dei prodotti, ritardandone il download fino allo scroll effettivo dell'utente.

- **Assenza di un Foglio di Stile di Stampa (Print Stylesheet)**

- **Problema:** Il report ha evidenziato la mancanza di un file CSS dedicato alla stampa (*print stylesheet*), il che porta il browser ad applicare le stesse regole di visualizzazione dello schermo desktop quando l'utente tenta di stampare la pagina o salvarla in PDF. Questo si traduce nella stampa su carta di elementi interattivi o grafici del tutto inutili (come menu di navigazione, pulsanti di azione, barre di ricerca, cursori del carosello e animazioni), in un eccessivo spreco di inchiostro o toner dovuto alla resa di sfondi colorati, gradienti o immagini decorative prive di valore informativo sulla carta, e in impaginazioni errate o troncate causate dall'inadeguatezza del layout a griglia dello schermo desktop rispetto alle dimensioni fisse dei fogli A4.
- **Soluzione:** Per risolvere l'assenza del *print stylesheet*, è stato creato un foglio di stile dedicato (`print.css`), richiamato in tutte le JSP tramite `media="print"`. Questo script impone un layout adattato alle dimensioni fisiche di un foglio A4, forza sfondi trasparenti con testo nero per risparmiare inchiostro, e nasconde completamente elementi interattivi e non testuali (come menu, pulsanti e footer).

• Mancanza di Direttive di Caching (Caching Headers)

- **Problema:** L'analisi ha evidenziato che la quasi totalità delle risorse statiche dell'applicazione (file CSS in `css/`, file JS in `functions/` e immagini in `img/`) venivano servite senza le opportune intestazioni HTTP di caching (come `Expires` o `Cache-Control`). Questa lacuna implica che ad ogni caricamento della pagina o navigazione verso una nuova area del sito, il browser è costretto a richiedere ex-novo le stesse risorse statiche (es. `general.css`, `general.js` e il logo aziendale). Questo produce un inutile sovraccarico di connessioni TCP e un elevato volume di traffico di rete sul server Tomcat e sull'infrastruttura di rete, con conseguente e facilmente evitabile dispendio energetico.
- **Soluzione:** Per ovviare alla mancanza di direttive di caching, è stato sviluppato un filtro servlet globale (`CachingFilter`) che intercetta le richieste dirette alle risorse statiche e inietta l'intestazione HTTP `Cache-Control: public, max-age=31536000, immutable` (insieme all'header `Expires`). Questa stessa logica è stata estesa anche al gestore di file dinamici `FileServlet`, assicurando che, dopo il primo download, il browser legga sempre gli asset dalla cache locale a costo energetico e di rete nullo.

• Mancata Compressione GZIP delle Risorse

- **Problema:** È stato riscontrato che le risorse testuali (come HTML/JSP, CSS, JS e favicon) venivano trasferite senza alcuna compressione. Questo costringeva a trasferire inutilmente l'intero peso non compresso dei file sulla rete, sprecando banda e incrementando l'energia assorbita dall'infrastruttura globale per l'instradamento dei pacchetti IP.
- **Soluzione:** Per abbattere il peso dei file trasferiti, è stato introdotto il `CompressionFilter`. Questo componente verifica il supporto del browser tramite l'header `Accept-Encoding: gzip` e comprime al volo le risorse testuali (JSP, CSS, JS, ecc.) incapsulando la risposta HTTP in un `GZIPOutputStream`, riducendo fino all'80% la quantità di byte scambiati sull'infrastruttura di rete globale.

• Errore HTTP 404 per il Favicon

- **Problema:** Durante l'audit è stato rilevato un errore HTTP 404 causato dal tentativo del browser di scaricare l'icona del sito (`favicon.ico`) all'indirizzo root, una risorsa originariamente assente. Gli errori di richiesta HTTP rappresentano una

forma di spreco energetico passivo, in quanto il server Tomcat e l'infrastruttura di rete elaborano e instradano la richiesta e la risposta di errore inutilmente, consumando risorse computazionali senza restituire alcun contenuto utile.

- **Soluzione:** Per risolvere l'errore passivo HTTP 404, è stata generata una favicon standard (`favicon.ico`) di 32x32 pixel a partire dal logo aziendale e posizionata nella root del progetto. In questo modo, ogni richiesta automatica del browser riceve correttamente l'asset grafico, eliminando del tutto le inutili risposte di errore e lo spreco di risorse sul server Tomcat.

- **Errori HTTP 500 per la Directory Immagini**

- **Problema:** È stato riscontrato un errore HTTP 500 in corrispondenza del percorso radice delle immagini (`/img/`). In assenza di un'immagine specifica, il servlet tentava di mappare la richiesta su una directory fisica sollevando un'eccezione non gestita sul server Tomcat, con conseguente generazione di costosi errori interni che dissipavano capacità di calcolo.
- **Soluzione:** Il difetto relativo alla mancata gestione del percorso root della cartella immagini (`/img/`) è stato corretto inserendo nel servlet `FileServlet.java` un controllo esplicito (`file.isDirectory()`). Questo blocco preventivo intercetta le richieste ambigue e restituisce un più corretto HTTP 404 Not Found, evitando le pesanti eccezioni non gestite (HTTP 500) all'interno dell'applicazione.

- **Trasmissione di Cookie nelle Risorse Statiche**

- **Problema:** Tutte le risorse statiche (CSS, JS, immagini e font) venivano trasmesse includendo l'header di richiesta `Cookie` contenente il token di sessione `JSESSIONID` (circa 0.9 KB di overhead per richiesta). Trattandosi di asset pubblici e immutabili, lo scambio continuo di queste informazioni di sessione rappresenta uno spreco ingiustificato di banda di rete.
- **Soluzione:** Per eliminare il passaggio dei cookie durante il download degli asset statici (es. `JSESSIONID`), è stata implementata una mappatura dinamica dell'host nelle JSP (che alterna `localhost` e `127.0.0.1` in modo da trattare le richieste statiche come cross-origin) associata all'attributo `crossorigin="anonymous"`. Contemporaneamente, il `CachingFilter` è stato configurato per inviare intestazioni CORS adeguate (`Access-Control-Allow-Origin: *`), disaccoppiando così i cookie dalle risorse.

- **Eccessiva Complessità del DOM (DOM Size)**

- **Problema:** L'utilizzo di container nidificati ridondanti (come quelli derivati dallo stile Bootstrap non ottimizzato) nelle griglie dinamiche aumentava inutilmente la complessità dell'albero DOM. Un DOM inutilmente profondo comporta un maggiore carico computazionale per la CPU del client durante le fasi di parsing, layouting e rendering.
- **Soluzione:** Per arginare l'eccessiva complessità del DOM e minimizzare l'impatto sul client, i container nidificati ridondanti di Bootstrap nei cicli JSP sono stati sostituiti con tag semantici HTML5 (come `<article>`), riducendo il numero totale di nodi e alleggerendo notevolmente il parsing della pagina anche durante i test di carico (es. 50 prodotti contemporanei).

- **Mancato Utilizzo del Protocollo HTTP/2**

- **Problema:** L’analisi ha segnalato che la maggior parte delle risorse veniva trasferita tramite il protocollo HTTP/1.1 anziché il più efficiente HTTP/2, il quale ridurrebbe l’overhead di rete grazie a multiplexing, compressione HPACK degli header e server push, evitando l’apertura di connessioni TCP ridondanti e dispendiose.
- **Soluzione:** L’assenza del protocollo HTTP/2 è stata identificata come una limitazione infrastrutturale dell’ambiente di sviluppo, in quanto i browser moderni richiedono necessariamente una connessione TLS valida (HTTPS) non compatibile con i normali certificati auto-firmati locali. L’adozione del protocollo è stata pertanto demandata all’ambiente di produzione, in cui un reverse proxy (es. Nginx o Caddy) configurerà dinamicamente la terminazione TLS permettendo a Tomcat di beneficiare delle ottimizzazioni senza modifiche applicative.

3.3 Validazione dell’Impatto del Refactoring

Di seguito viene presentato il confronto dettagliato delle metriche rilevate:

I dati dimostrano l’efficacia del refactoring:

- **aggiuntaProdotto.jsp:** Il peso della pagina è stato ridotto del 41% (da 187,7 KB a 110,8 KB) e le richieste HTTP sono scese da 12 a 11 grazie alla rimozione della libreria jQuery (sostituita con chiamate Vanilla JS native) e all’introduzione del filtro di compressione GZIP a livello globale.
- **carrello.jsp:** È stata rimossa la dipendenza da jQuery riscrivendo lo script `carrello.js` e appiattendendo la struttura del DOM nel ciclo di iterazione dei prodotti. Questo intervento ha ridotto la dimensione della pagina da 186,1 KB a 111,1 KB (circa 40% di risparmio in peso).
- **categorie.jsp:** La rimozione delle librerie CDN non utilizzate ha ridotto le richieste HTTP da 11 a 8. La semplificazione strutturale ha diminuito i nodi DOM da 66 a 64 e il peso della pagina è calato del 42,8% (da 189,5 KB a 108,4 KB), elevando il punteggio EcoIndex a **92,0**.
- **chiSiamo.jsp:** La totale rimozione del codice JavaScript non necessario e la semplificazione semantica dei paragrafi di testo hanno abbattuto il peso da 190,8 KB a 109,3 KB (risparmio del 42,7%) e ridotto le richieste HTTP da 11 a 8, aumentando lo score EcoIndex a **93,0**.
- **confronta.jsp:** La riscrittura in Vanilla JS dello script `confronta.js` e l’eliminazione dei tag wrapper extra hanno ridotto la dimensione di rete da 186,7 KB a 109,3 KB (41,4% di risparmio) e le richieste HTTP da 11 a 9.
- **login.jsp:** L’eliminazione delle dipendenze pesanti CDN (FontAwesome e jQuery) e la compressione del logo aziendale hanno abbattuto il peso del 96,7% (da 98,5 KB a soli 3,3 KB) e ridotto le richieste HTTP da 8 a 5, portando lo score EcoIndex a **96,0**.
- **modificaProdotto.jsp:** La riscrittura dello script `modificaProdotto.js` in puro Vanilla JS e l’ottimizzazione del rendering del form hanno ridotto il peso del 78,7% (da 100,0 KB a 21,3 KB), mantenendo un ottimo score EcoIndex pari a **95,0**.
- **paginaProdotto.jsp:** Il passaggio al JavaScript nativo e la semplificazione delle card delle azioni hanno ridotto il peso della pagina del 43,0% (da 196,7 KB a 112,2 KB) e ridotto le richieste da 12 a 10.

Pagina / Vista	Ramo	Peso (KB)	Nodi DOM	Richieste	Grade	EcoIn
aggiuntaProdotto.jsp	Baseline	187.652	82	12	A	
	Optimized	110.808	87	11	A	
carrello.jsp	Baseline	186.145	49	11	A	
	Optimized	111.053	64	11	A	
categorie.jsp	Baseline	189.503	66	11	A	
	Optimized	108.370	64	8	A	
chiSiamo.jsp	Baseline	190.777	52	11	A	
	Optimized	109.280	49	8	A	
confronta.jsp	Baseline	186.702	89	11	A	
	Optimized	109.268	143	9	A	
login.jsp	Baseline	98.541	22	8	A	
	Optimized	3.265	20	5	A	
modificaProdotto.jsp ?prodotto=100	Baseline	100.026	6	16	A	
	Optimized	21.345	8	16	A	
paginaProdotto.jsp ?prodotto=100	Baseline	196.666	100	12	A	
	Optimized	112.233	125	10	A	
paginaUtente.jsp	Baseline	190.827	81	13	A	
	Optimized	110.714	92	10	A	
registrazione.jsp	Baseline	104.861	26	9	A	
	Optimized	3.399	23	5	A	
visualizzaOrdine.jsp ?idOrdine=1&cod=0	Baseline	186.503	44	13	A	
	Optimized	110.013	48	11	A	
visualizzaOrdineAdmin.jsp ?idOrdine=1&cod=0	Baseline	96.288	6	16	A	
	Optimized	19.094	8	15	A	
wishlist.jsp (6 prodotti)	Baseline	186.305	87	12	A	
	Optimized	110.455	78	11	A	
showProdotti.jsp (50 prodotti)	Baseline	634.085	85	18	A	
	Optimized	168.200	345	13	A	
index.jsp (50 prodotti)	Baseline	797.183	117	21	A	
	Optimized	284.108	107	19	A	
dashboard.jsp	Baseline	796.840	118	20	A	
	Optimized	283.924	107	18	A	

Table 3.1: Confronto delle metriche ambientali prima (Baseline) e dopo (Optimized) il refactoring.

- **paginaUtente.jsp**: La rimozione di jQuery e dei tag di formattazione non semantici ha abbattuto il peso del 42,0% (da 190,8 KB a 110,7 KB) e ridotto le richieste HTTP da 13 a 10.
- **registrazione.jsp**: Seguendo lo stesso approccio di **login.jsp**, sono state eliminate le dipendenze CDN inutilizzate e consolidato il layout, riducendo il peso del 96,8% (da 104,9 KB a 3,4 KB), le richieste HTTP da 9 a 5, e portando lo score EcoIndex a **96,0**.
- **visualizzaOrdine.jsp**: La rimozione di script non utilizzati e l'appiattimento del DOM hanno ridotto il peso del 41,0% (da 186,5 KB a 110,0 KB) e le richieste HTTP da 13 a 11.

- **visualizzaOrdineAdmin.jsp**: Riscritto interamente in Vanilla JS, ha permesso di abbattere il peso del 80,2% (da 96,3 KB a 19,1 KB) e ridurre le richieste HTTP da 16 a 15, mantenendo il punteggio EcoIndex stabile a **95,0**.
- **wishlist.jsp**: La rimozione di jQuery e lo switch a miniature compresse con caricamento lazy hanno ridotto il peso del 40,7% (da 186,3 KB a 110,5 KB), i nodi DOM da 87 a 78 e le richieste HTTP da 12 a 11, aumentando il punteggio EcoIndex a **91,0**.
- **showProdotti.jsp**: Il passaggio all'utilizzo esclusivo delle miniature dei prodotti compresse in formato WebP ha ridotto drasticamente il peso della griglia da 634,1 KB a 168,2 KB (risparmio del 73,5%) e le richieste da 18 a 13, nonostante la pagina sia stata sottoposta a stress-test caricando contemporaneamente ben 50 prodotti.
- **index.jsp**: La ristrutturazione semantica del menu e l'ottimizzazione del carosello principale hanno abbattuto il peso della landing page da 797,2 KB a 284,1 KB (risparmio del 64,4%) e ridotto le richieste HTTP da 21 a 19, innalzando lo score EcoIndex a **88,0**.
- **dashboard.jsp**: Grazie all'introduzione del `SessionInjectionFilter` (che ha consentito l'accesso all'area protetta per il test), è stato possibile effettuare la misurazione che evidenzia un netto miglioramento rispetto ai valori stimati di baseline: il peso della pagina è sceso da 796,8 KB a 283,9 KB (risparmio del 64,4%), i nodi DOM sono calati da 118 a 107, e le richieste HTTP si sono ridotte da 20 a 18, portando il punteggio EcoIndex da 84,0 a **88,0** (Grade A).

3.3.1 Quantificazione dell'Ottimizzazione Globale

L'efficacia complessiva del piano di refactoring e ottimizzazione ecologica del front-end di 2Chance può essere scientificamente quantificata aggregando e confrontando le metriche totali delle 16 pagine analizzabili in entrambi gli scenari:

1. **Riduzione del Payload di Rete (Byte Risparmiati)**: Il peso complessivo trasferito per il caricamento delle 16 pagine del portale è sceso da **4,82 MB** (4.818.904 byte) della versione Baseline a soli **1,87 MB** (1.865.529 byte) della versione Ottimizzata. Questo rappresenta una riduzione netta del **61,3%** dei dati trasmessi sulla rete globale.
2. **Riduzione delle Richieste HTTP**: Il numero totale di richieste HTTP necessarie per caricare l'intero portale è sceso da **214** a **182**, registrando un risparmio netto del **15,0%**. Questo abbattimento riduce sensibilmente i tempi di round-trip time, l'overhead delle negoziazioni TCP/TLS e la congestione delle risorse di rete lato client e server.
3. **Abbattimento delle Emissioni di Gas Serra (gCO₂e)**: Attraverso l'algoritmo standard di GreenIT, le emissioni totali stimate per il caricamento completo del portale sono scese da **18,94 gCO₂e** della versione Baseline a **18,84 gCO₂e** della versione Ottimizzata. Proiettato su decine di migliaia di visualizzazioni mensili, questo risparmio cumulativo riduce drasticamente l'impronta di carbonio complessiva dell'applicativo generata dal carico computazionale dei datacenter e dall'instradamento di rete.
4. **Riduzione del Consumo Idrico Equivalente (cl)**: Il consumo idrico virtuale (legato al raffreddamento dei server e alla produzione di energia elettrica) è sceso proporzionalmente da **28,41 cl** a **28,26 cl** per ciclo di caricamento.
5. **Efficienza del DOM sotto Carico**: Nelle griglie dinamiche (come `showProdotti.jsp` e `index.jsp`), nonostante il numero di articoli pre-caricati sia stato quadruplicato (da 12

a 50 articoli) per simulare un carico di stress reale, l'appiattimento strutturale del markup e l'introduzione del Lazy Loading hanno evitato la proliferazione incontrollata dei nodi DOM e il pre-caricamento di immagini non visibili, preservando la CPU e la memoria RAM del client.

Chapter 4

Monitoraggio Licenze Open Source

4.1 Analisi della Compliance e dei Rischi Legali (FOSSA)

4.1.1 Sostenibilità Legale ed Economica: Software Composition Analysis e Conformità delle Licenze

La sostenibilità di un artefatto software non è misurabile esclusivamente in termini di Watt consumati o emissioni di anidride carbonica; essa è profondamente legata alla vitalità economica a lungo termine del prodotto e alla mitigazione dei rischi legali. Un software che possiede un'architettura ambientalmente impeccabile, ma il cui codice viola sistematicamente le licenze intellettuali di terze parti, è destinato a un inevitabile fallimento economico a causa di cause legali, sanzioni e ingiunzioni di blocco della distribuzione.

4.1.2 Monitoraggio delle Licenze Open Source tramite FOSSA

FOSSA è una piattaforma enterprise dedicata alla governance delle dipendenze e alla compliance delle licenze open source. Essa non si limita a leggere i metadati manifesti nei file `.pom`, ma offre una tecnologia di scansione full-text (Audit-Grade License Scanning) capace di analizzare il codice sorgente grezzo delle dipendenze con una precisione dichiarata del 99.8%. Questo meccanismo permette di rilevare intestazioni di copyright, identificare stralci di codice copiato in modo improprio ed estrarre i vincoli normativi anche da varianti di licenze alterate o non standard (es. licenze MIT con clausole di attribuzione modificate).

La risoluzione dei conflitti legali identificati da FOSSA, che spesso si traducono nella rimozione di una libreria con una licenza inadatta per sostituirla con un'alternativa funzionalmente equivalente ma dotata di licenza permissiva (MIT o Apache), rappresenta un intervento che, pur non modificando le funzionalità visibili all'utente, consolida l'integrità strutturale ed economica del software, rendendolo pronto per un'ipotetica adozione su scala enterprise.

Per garantire una sostenibilità legale ferrea, l'integrazione di FOSSA all'interno del progetto 2Chance avverrà sfruttando un approccio moderno Cloud-Native tramite la CLI dedicata e la piattaforma Web. L'ecosistema di sviluppo Java, sul quale si erge l'intera piattaforma 2Chance, si appoggia strutturalmente su repository immensi di dipendenze esterne (Maven Central). Ogni libreria dichiarata nel file `pom.xml` eredita a sua volta decine di librerie transitive, creando un albero di dipendenze labirintico e opaco. Data la complessità degli alberi delle dipendenze in Java, la CLI di FOSSA analizzerà autonomamente il `pom.xml` per costruire il grafo completo e inviarlo al Cloud, mappando accuratamente sia le dipendenze dirette (dichiarate esplicitamente dagli sviluppatori) sia quelle transitive (ereditate indirettamente). L'obiettivo strategico di questa imponente operazione di conformità è declinato su tre livelli:

1. **Identificazione Sistematica del Rischio Copyleft:** Il nucleo del problema legale

risiede nella classificazione delle licenze in due macro-categorie: permissive (es. MIT, Apache 2.0) e copyleft (es. GPL v2/v3, AGPL). Le licenze permissive impongono vincoli minimi (principalmente la citazione), mentre le licenze copyleft sono “virali”: l’inclusione, anche accidentale, di una singola libreria GPL all’interno di un progetto proprietario closed-source può obbligare legalmente gli autori a rilasciare gratuitamente l’intero codice sorgente dell’applicazione, annullando ogni potenziale di monetizzazione e protezione del segreto industriale. FOSSA mapperà l’intero albero delle dipendenze ed evidenzierà con alert critici eventuali violazioni di policy legate alla presenza non desiderata di licenze copyleft.

2. **Automazione Assoluta degli Obblighi di Attribuzione:** La stragrande maggioranza delle licenze open source come Apache 2.0 impone obblighi legali stringenti per la distribuzione, tra cui la conservazione inalterata dei file di NOTA (NOTICE) e la generazione di documenti di attribuzione visibili all’utente finale. FOSSA dispone di funzionalità di Automated Attributions, in grado di compilare e aggiornare dinamicamente e automaticamente l’elenco di tutte le licenze e i crediti richiesti, sollevando il team di sviluppo da una prassi manuale, tediosa e ad altissimo rischio di errore umano.
3. **Generazione della Software Bill of Materials (SBOM):** Una compliance proattiva richiede la trasparenza della catena di approvvigionamento. FOSSA favorirà l’esportazione automatizzata di una SBOM (Distinta Base del Software) aderente agli standard industriali come SPDX o CycloneDX. La fornitura di una SBOM chiara è un requisito sempre più pressante nei bandi di gara governativi e costituisce un pilastro per l’ottenimento di certificazioni aziendali internazionali.

4.1.3 Esecuzione dell’Analisi FOSSA

L’analisi è stata eseguita adottando un approccio moderno e integrato che fa cooperare la CLI ufficiale di FOSSA (eseguibile localmente o in ambiente CI/CD) con il potente motore di calcolo della piattaforma cloud (FOSSA Web App).

Il nuovo approccio si basa sull’autenticazione sicura della CLI verso il servizio cloud tramite una API Key dedicata (isolata in un file `.fossa_token` e gestita tramite i *Secrets* in GitHub Actions). Questo supera i severi limiti di un approccio puramente offline, che costringerebbe all’uso combinato di plugin ausiliari limitati alla sola lettura dei metadati senza ispezione del codice sorgente.

L’esecuzione coordinata avviene nei seguenti passaggi:

1. **Risoluzione del Grafo (CLI):** La FOSSA CLI, invocata tramite il comando `fossa analyze`, identifica automaticamente il progetto Maven parsando il file `pom.xml` e risolve l’intero albero delle dipendenze (dirette e transitive). L’output di questa operazione genera due artefatti diagnostici locali nella directory `src/fossa`: il payload grezzo JSON `fossa-analyze-output.json` e il log testuale `fossa-scan-summary.txt`.
2. **Sincronizzazione Cloud:** La CLI comunica con i server remoti, inviando il payload JSON del grafo strutturato delle dipendenze in tempo reale alla dashboard di riferimento.
3. **Analisi Profonda (Web App):** La piattaforma cloud prende in carico il grafo, scarica il codice sorgente grezzo di ciascuna libreria ed esegue una scansione *Audit-Grade*. In questa fase vengono analizzati anche i *copyright headers* e il codice sorgente annidato, scoprendo violazioni copyleft che non emergerebbero mai nei semplici manifesti POM.

4.1.4 Albero delle Dipendenze e Inventario delle Licenze

FOSSA CLI ha mappato in autonomia l'intero albero delle dipendenze del progetto `groupId:2chance:war:` identificando **15 dipendenze dirette** (dichiarate esplicitamente nel `pom.xml`) e **8 dipendenze transitive** (ereditate indirettamente), per un totale di **23 artefatti**. A supporto di questa mappatura visiva, è stato inoltre estratto il file `dependency-tree.txt` tramite il comando `mvn dependency:tree`, che fornisce una chiara gerarchia testuale di chi importa cosa e con quale *scope*.

Table 4.1: Inventario delle licenze delle dipendenze del progetto 2Chance.

Dipendenza	Scope	Licenza	Categoria
commons-io:commons-io 2.14.0	compile	Apache-2.0	Permissiva
org.json:json 20231013	compile	Public Domain	Permissiva
org.apache.tomcat:tomcat-jdbc 9.0.111	compile	Apache-2.0	Permissiva
org.apache.tomcat:tomcat-juli 9.0.111	compile	Apache-2.0	Permissiva
com.google.protobuf:protobuf-java 4.31.1	compile	BSD-3-Clause	Permissiva
org.apache.taglibs:taglibs-standard-1.2.5	compile	Apache-2.0	Permissiva
org.apache.taglibs:taglibs-standard-1.2.5	compile	Apache-2.0	Permissiva
org.pitest:pitest-parent 1.1.10	compile	Apache-2.0	Permissiva
net.sf.jopt-simple:jopt-simple 5.0.4	compile	MIT	Permissiva
org.apache.commons:commons-math3 3.6.1	compile	Apache-2.0	Permissiva
com.mysql:mysql-connector-j 9.5.0	compile	GPLv2 + FOSS Exc.	Copyleft+Exc.
org.openjdk.jmh:jmh-core 1.37	compile	GPLv2 + CE	Copyleft+Exc.
javax.servlet:javax.servlet-api 4.0.1	provided	CDDL + GPLv2 + CE	Copyleft+Exc.
org.openjdk.jmh:jmh-generator-asm 1.37	provided	GPLv2 + CE	Copyleft+Exc.
org.junit.jupiter:junit-jupiter-api 5.9.1	test	EPL-2.0	Copyleft Debole
org.junit.jupiter:junit-jupiter-engine 5.9.1	test	EPL-2.0	Copyleft Debole
org.junit.jupiter:junit-jupiter-params 5.9.1	test	EPL-2.0	Copyleft Debole
org.junit.platform:junit-platform-commons 1.9.1	test	EPL-2.0	Copyleft Debole
org.junit.platform:junit-platform-engine 1.9.1	test	EPL-2.0	Copyleft Debole
org.mockito:mockito-core 5.11.0	test	MIT	Permissiva

Continua nella pagina successiva

Dipendenza	Scope	Licenza	Categoria
org.mockito:mockito-junit-jupiter 5.11.0	test	MIT	Permissiva
net.bytebuddy:byte-buddy 1.14.12	test	Apache-2.0	Permissiva
net.bytebuddy:byte-buddy-agent 1.14.12	test	Apache-2.0	Permissiva
org.objenesis:objenesis 3.3	test	Apache-2.0	Permissiva
org.opentest4j:opentest4j 1.2.0	test	Apache-2.0	Permissiva
org.apiguardian:apiguardian-apitest 1.1.2		Apache-2.0	Permissiva

Ogni dipendenza ha uno *scope* definito che è fondamentale ai fini dell'analisi legale. In ambiente Maven, i principali *scope* rilevati nel progetto si dividono in:

- **compile**: le dipendenze vengono incluse fisicamente nel pacchetto WAR distribuito, esponendo direttamente l'intero progetto ai vincoli della loro licenza.
- **provided**: le dipendenze sono fornite dal container a runtime (es. Tomcat) e non vengono pacchettizzate nell'artefatto finale, mitigando drasticamente il rischio legale di distribuzione.
- **test**: le dipendenze sono utilizzate esclusivamente durante la fase di testing locale. Non venendo mai consegnate all'utente finale, sono di fatto esenti da vincoli o obblighi di distribuzione copyleft.

La Tabella 4.1 presenta l'inventario esaustivo estratto dalla scansione. L'analisi rivela una distribuzione quantitativa molto favorevole delle licenze, che possono essere classificate in quattro macro-categorie di rischio legale (riassunte statisticamente nella Tabella 4.2):

- **Permissive** (18 dipendenze, 69,2% del totale): rischio nullo (es. MIT, Apache 2.0). Impongono vincoli minimi, come la conservazione dell'attribuzione del copyright originale, e consentono la libera integrazione in software proprietario chiuso. Rappresentano la netta maggioranza dell'ecosistema di 2Chance.
- **Copyleft Debole** (5 dipendenze, 19,2% del totale): rischio basso (es. EPL-2.0, LGPL). L'obbligo virale di rilasciare il codice sorgente è generalmente limitato alle sole modifiche effettuate direttamente sulla libreria stessa, senza intaccare il codice del progetto principale che ne fa uso.
- **Copyleft con Eccezioni** (3 dipendenze, 11,5% del totale): rischio medio (es. GPLv2 con Classpath Exception o FOSS Exception). Il rischio virale intrinseco della licenza GPL pura è esplicitamente disinnescato da clausole aggiuntive che ne consentono il linking dinamico o l'integrazione con altre licenze open source compatibili.
- **Copyleft senza Eccezioni** (0 dipendenze, 0,0% del totale): rischio critico (es. GPL/AGPL pura). Questa categoria imporrebbe il rilascio open source dell'intero codice proprietario. L'assenza totale di queste licenze nel progetto rappresenta il dato più rilevante dell'analisi, confermando che **nessuna violazione critica è stata rilevata**.

Tipologia di Licenza	Conteggio	Percentuale
Apache-2.0	13	50,0%
MIT	3	11,5%
BSD-3-Clause	1	3,8%
Public Domain	1	3,8%
Totale Permissive	18	69,2%
Eclipse Public License 2.0 (EPL-2.0)	5	19,2%
GPLv2 + Universal FOSS Exception	1	3,8%
GPLv2 + Classpath Exception	2	7,7%
Totale Copyleft/Copyleft con Eccezione	8	30,8%
GPL/AGPL pura (senza eccezioni)	0	0,0%

Table 4.2: Distribuzione delle tipologie di licenza nell'albero delle dipendenze di 2Chance.

Livello di Rischio	N.	Dettaglio
Critico (GPL/AGPL pura)	0	Nessuna violazione copyleft pura rilevata
Medio (Copyleft + Eccezione)	2	<code>mysql-connector-j</code> , <code>jmh-core</code>
Basso (Scope provided/test)	2	<code>jmh-generator-annprocess</code> , <code>javax.servlet-api</code>
Nessuno (Permissiva/Test)	22	Tutte le rimanenti dipendenze

Table 4.3: Sintesi della valutazione del rischio copyleft per il progetto 2Chance.

4.2 Integrazione Cloud e Continuous Compliance

Per superare i limiti dell'analisi esclusivamente locale e statica basata sul `license-maven-plugin`, che si limita alla lettura dei metadati senza ispezionare il codice (producendo semplicemente il manifesto `licenses.xml` e scaricando i testi in chiaro all'interno della cartella `licenses/`), è stata effettuata la transizione a **FOSSA Web Service**. Questo approccio con la scansione cloud avanzata (*Policy Scan*), garantisce non solo la risoluzione automatica delle licenze, ma anche uno scan approfondito del codice sorgente delle dipendenze per intercettare violazioni nascoste (Audit-Grade Scanning) e la verifica automatica delle policy aziendali.

Per l'integrazione di FOSSA CLI e FOSSA Web Service è stato registrato un account sulla piattaforma FOSSA (<https://app.fossa.com>), poi è stata generata una API Key dedicata dalla dashboard web di FOSSA. All'interno dello workspace di 2chance locale, la chiave è stata salvata ed isolata all'interno di un file `.fossa_token`, ignorato dal Versioning Control System tramite `.gitignore`. Sulla piattaforma GitHub, la chiave è stata memorizzata in modo sicuro tra i *Repository Secrets* (`FOSSA_API_KEY`).

La piattaforma FOSSA Web Service con l'esecuzione della scansione cloud avanzata (*Policy Scan*), ha applicato un set di policy di default molto più rigoroso di un'analisi locale basata sulla

mera ispezione dei manifesti Maven, restituendo un totale di **10 issue** di licenza, categorizzati in due livelli di gravità:

- **Flagged:** Rappresentano licenze identificate come potenzialmente problematiche o ambigue (ad esempio, copyleft deboli o con eccezioni). Non causano necessariamente un blocco immediato della pipeline, ma richiedono obbligatoriamente una *review* manuale da parte del team per certificare che il contesto d'uso specifico (es. linking dinamico, ambito di test) non violi i termini legali.
- **Denied:** Rappresentano violazioni critiche e dirette delle policy (tipicamente licenze copyleft forti come la GPLv2 pura o eccezioni decadute). Causano il blocco automatico della build (*Fail on Violation*) e richiedono un intervento correttivo drastico (rimozione della libreria) oppure la creazione esplicita di un'eccezione motivata.

4.2.1 Risultati della Scansione (Policy Scan) e Valutazione del Rischio Legale per Macro-Aree Architettureali

Dall'incrocio tra la topologia del grafo delle dipendenze e l'inventario delle licenze ottenuti con FOSSA CLI e FOSSA Web Service emergono diverse macro-aree architetturali, ciascuna con un differente impatto teorico e pratico sul profilo di rischio legale del software. Di seguito si presenta la valutazione dettagliata per ciascuna area, integrando l'analisi delle specifiche dipendenze e delle policy flaggate da FOSSA:

- **Infrastruttura Web e Servlet:** Include le API fondamentali fornite dall'Application Server (Apache Tomcat) a runtime. L'assenza fisica di queste librerie nel pacchetto WAR distribuito abbate il rischio di violazioni copyleft.
 - `javax.servlet:javax.servlet-api:4.0.1` (**CDDL + GPLv2 con Classpath Exception**):
 - * **Scope:** provided
 - * **Dati FOSSA Web Service:** Tipo: Maven | Profondità: Transitive | Licenze rilevate: *Flagged: GPL-2.0-with-classpath-exception, Flagged: GPL-2.0-only, Flagged: CDDL-1.1*
 - * **Categoria di Rischio:** Permissiva / Copyleft inibito dallo scope (Rischio Nullo)
 - * **Utilizzo:** API fondamentali fornite dall'Application Server (Apache Tomcat) a runtime.
 - * **Soluzione / Mitigazione:** La libreria ha sollevato issue in FOSSA poiché la GPL-2.0 pura imporrebbe vincoli virali. Come soluzione didattica, essendo fornita dal container a runtime, la libreria possiede scope **provided** e non viene inclusa nel pacchetto finale, permettendo di marcare i flag come "Approvati". Nello scenario commerciale, la soluzione è identica: le dipendenze **provided** non innescano obblighi copyleft distributivi e il flag verrebbe approvato legalmente.
- **Connettività Database e Persistenza:** Questi artefatti sono vitali per l'applicazione e vengono compressi direttamente nel pacchetto finale distribuito. Rappresentano il vettore di rischio legale più critico.
 - `com.mysql:mysql-connector-j:9.5.0` (**GPLv2 + Universal FOSS Exception v1.0**):
 - * **Scope:** compile

- * **Dati FOSSA Web Service:** Tipo: Maven | Profondità: Transitive | Licenze rilevate: *Flagged: GPL-2.0-only, Flagged: LGPL-2.1-only, Flagged: LGPL-2.0-or-later, Denied: gpl-2.0 WITH universal-foss-exception-1.0*
- * **Categoria di Rischio:** Copyleft forte con eccezione (Rischio Medio)
- * **Utilizzo:** Connettore vitale per l'interazione dell'applicazione con il database MySQL.
- * **Soluzione / Mitigazione:** Bloccato di default da FOSSA a causa del copyleft forte (*Denied*). Come soluzione didattica per questo progetto open source, è stata applicata l'opzione "*Create Exception*" poiché la Universal FOSS Exception ne garantisce la compatibilità attuale. In uno scenario commerciale proprietario, la FOSS Exception decadrebbe esponendo l'azienda. In tal caso per la distribuzione closed-source, la libreria dovrebbe essere sostituita nel `pom.xml` con `org.mariadb.jdbc:mariadb-java-client` (funzionalmente equivalente, rilasciato sotto licenza più permissiva LGPL-2.1), oppure sostituita acquistando una licenza commerciale da Oracle.
- **org.apache.tomcat:tomcat-jdbc e dipendenze transitive (Apache-2.0 / BSD-3-Clause):**
 - * **Scope:** compile
 - * **Categoria di Rischio:** Licenze permissive (Rischio Nullo)
 - * **Utilizzo:** Pool di connessioni (`tomcat-juli`) e serializzazione (`protobuf-java`).
 - * **Soluzione / Mitigazione:** Nessun intervento sul codice necessario poiché le licenze sono sicure per l'uso commerciale. Tuttavia, per soddisfare gli obblighi imposti dalle licenze Apache-2.0, MIT e BSD-3-Clause, si raccomanda la **generazione di un documento di attribuzione formale** (`THIRD-PARTY-NOTICES.txt`) contenente i testi integrali scaricati, da includere nella distribuzione.
- **Utilità Generiche e Manipolazione Dati:** Raggruppa le librerie utilizzate trasversalmente nel codice.
 - **commons-io, org.json, taglibs (Apache 2.0 / MIT / Public Domain):**
 - * **Scope:** compile
 - * **Categoria di Rischio:** Licenze permissive / Pubblico dominio (Rischio Nullo)
 - * **Utilizzo:** Componenti storici e utilità generiche per la manipolazione dei dati.
 - * **Soluzione / Mitigazione:** L'inclusione nel WAR distribuito è sicura dal punto di vista commerciale. Nessun rischio copyleft è associato a queste librerie. L'unica azione operativa richiesta è l'inserimento nel documento formale `THIRD-PARTY-NOTICES.txt` per l'attribuzione.
- **Ecosistema di Testing e Mocking:** Dal punto de vista della compliance legale, questa vasta ramificazione dell'albero è intrinsecamente sicura grazie allo scope isolato.
 - **org.junit.jupiter (EPL-2.0):**
 - * **Scope:** test
 - * **Categoria di Rischio:** Copyleft debole isolato dallo scope (Rischio Nullo)
 - * **Utilizzo:** Framework principale per la scrittura e l'esecuzione dei test.
 - * **Soluzione / Mitigazione:** Lo scope *test* garantisce al livello di build system che Maven escluda questi file dall'artefatto di produzione. Questo meccanismo annulla preventivamente e alla radice ogni rischio di distribuzione copyleft.

- **org.mockito:mockito-core e dipendenze transitive (MIT / Apache 2.0):**
 - * **Scope:** test
 - * **Categoria di Rischio:** Licenze permissive (Rischio Nullo)
 - * **Utilizzo:** Strumenti di mocking (incluso `byte-buddy`) per l'isolamento dei test.
 - * **Soluzione / Mitigazione:** Componenti totalmente sicuri, resi ininfluenti sul WAR finale grazie allo scope di test. Nessun intervento necessario.
- **Strumenti di Benchmarking e Qualità:** L'albero fa emergere un'anomalia architetturale significativa: l'inclusione di framework diagnostici nel WAR finale di produzione è un errore che espone inutilmente l'azienda a vincoli legali.
 - **org.openjdk.jmh:jmh-core:1.37 (GPLv2 + Classpath Exception):**
 - * **Scope:** compile (errato architetturalmente)
 - * **Categoria di Rischio:** Copyleft forte con eccezione (Rischio Medio)
 - * **Utilizzo:** Strumento di benchmarking per profilazione energetica. Non ha utilità funzionale in produzione.
 - * **Soluzione / Mitigazione:** L'inclusione nel WAR distribuito introduce un rischio legale evitabile e un peso inutile. Un'analisi del codice ha confermato che le importazioni di JMH sono relegate ai soli file di benchmark. L'intervento correttivo applicato è stata la **modifica dello scope da compile a test** nel `pom.xml`. Questa azione elimina completamente la libreria e le sue dipendenze transitive (`jopt-simple`, `commons-math3`) dal WAR distribuito, riducendo significativamente l'impronta delle dipendenze copyleft e annullando totalmente il rischio associato alla licenza GPLv2 + CE senza alterare le funzionalità.
 - **org.openjdk.jmh:jmh-generator-annprocess:1.37 (GPLv2 + Classpath Exception):**
 - * **Scope:** provided
 - * **Dati FOSSA Web Service:** Tipo: Maven | Profondità: Transitive | Licenze rilevate: *Flagged: GPL-2.0-only, Flagged: GPL-2.0-with-classpath-exception*
 - * **Categoria di Rischio:** Copyleft inibito dallo scope (Rischio Nullo in distribuzione)
 - * **Utilizzo:** Processore di annotazioni per generare il codice JMH.
 - * **Soluzione / Mitigazione:** FOSSA ha applicato un blocco cautelativo poiché la licenza virale potrebbe impattare il progetto se distribuita. Come soluzione didattica, ha ricevuto un'approvazione manuale ("*Approve as Exception*"). In uno scenario commerciale, la soluzione aziendale non si affiderebbe ad approvazioni manuali per la GPL: imporrebbe un enforcing tecnologico rigoroso (assicurandosi tramite CI/CD che tutte le dipendenze di benchmarking abbiano categoricamente scope `test` o `provided`), rendendo l'inclusione accidentale tecnicamente impossibile.
 - **org.pitest:pitest-parent (Apache 2.0):**
 - * **Scope:** compile (errato architetturalmente)
 - * **Categoria di Rischio:** Licenza permissiva (Rischio Nullo)
 - * **Utilizzo:** Supporto al plugin di mutation testing `pitest-maven`.
 - * **Soluzione / Mitigazione:** Pur non ponendo rischi legali, l'uso come `compile` appesantisce inutilmente l'artefatto. Poiché nessun file Java dell'applicazione importa classi da `org.pitest`, l'intervento correttivo applicato, privo di rischi

funzionali, è consistito nel modificare lo scope a `test`, escludendolo dall'artefatto di produzione.

4.2.2 Generazione della Software Bill of Materials (SBOM)

Al fine di garantire la trasparenza della catena di approvvigionamento del software (*software supply chain*) ed adempiere alle moderne richieste di conformità legale, la generazione della **Software Bill of Materials (SBOM)** consente di descrivere in modo univoco e strutturato ciascun componente software incluso nel build, specificando per ognuno di essi: identificativi univoci, versione precisa, e classificazione delle licenze applicate.

Questi artefatti vengono generati dinamicamente dal cloud senza necessitare di ulteriori plugin Maven aggiuntivi nel `pom.xml`, grazie a FOSSA Web Service che offre funzionalità di esportazione native conformi agli standard industriali. Per il progetto 2Chance, è possibile generare ed esportare report dettagliati in formati strutturati quali:

- **CycloneDX (JSON/XML)**: Formato ottimizzato per la sicurezza e l'analisi automatica da parte di strumenti di Vulnerability Management esterni. Viene esportato il file `sbom.json`, contenente l'inventario completo e gli hash crittografici dei pacchetti.
- **SPDX**: Standard ISO ufficiale per la condivisione delle informazioni sulle licenze software. Questo viene esportato nel file `sbom.spdx` utilizzando la sintassi Tag:Value universalmente riconosciuta nell'industria.

4.3 Integrazione di FOSSA nel pipeline CI/CD

Per garantire un monitoraggio proattivo e continuo, è stata configurata l'API key di FOSSA Web Service nel workflow GitHub Actions di 2chance. Ciò abilita la scansione automatizzata di ogni dipendenza ereditata dal `pom.xml` ad ogni commit, la generazione di SBOM in formato SPDX/CycloneDX e un quality gate che, causando il fallimento della pipeline CI/CD, blocca automaticamente le pull request. In questo modo si impedisce il merge nel branch di produzione di codice legalmente rischioso, ossia quello che introduce dipendenze con licenze non conformi alla policy del progetto (come copyleft non consentite o violazioni degli obblighi di attribuzione).

Il file di workflow `main.yml` è stato esteso introducendo lo step di *License Compliance Scan* subito dopo l'analisi delle vulnerabilità (Snyk). Avvalendosi dell'azione ufficiale `fossas/fossa-action@v1`, ogni nuova *Pull Request* e operazione di *Push* invia automaticamente il grafo delle dipendenze alla piattaforma cloud di FOSSA.

L'intero progetto, essendo destinato a scopi puramente didattici e non prevedendo alcuna distribuzione commerciale, ha consentito un approccio flessibile alla risoluzione di queste segnalazioni. Tuttavia, questi risultati confermano il valore cruciale dell'integrazione CI/CD e di uno strumento di *Software Composition Analysis* moderno: sebbene le licenze individuate si siano rivelate gestibili nel contesto didattico specifico di 2Chance, in un contesto industriale reale la pipeline FOSSA avrebbe efficacemente sventato il rilascio in produzione del connettore MySQL, salvando l'azienda da gravissime implicazioni legali e ingiunzioni di distribuzione.

4.3.1 Automazione degli Obblighi di Attribuzione

Per soddisfare pienamente il secondo obiettivo di conformità legale, ovvero l'automazione assoluta degli obblighi di attribuzione, l'intero processo è stato delegato alla suite di *Reporting* nativa di FOSSA Web Service che compila e aggiorna dinamicamente la lista completa delle dipendenze, delle licenze associate e dei testi di copyright necessari ricavandoli dall'analisi profonda del codice sorgente. Questa fase culmina nella generazione del documento formale

di attribuzione `THIRD-PARTY-NOTICES.txt` (da distribuire nel progetto a garanzia di compliance) e della sua controparte web interattiva `third-party-report.html`. L'adozione di questa generazione automatizzata in cloud garantisce che la documentazione sulle licenze sia sempre perfettamente allineata con l'ultimo commit validato tramite la pipeline CI/CD, sollevando il team di sviluppo da controlli manuali inclini all'errore umano e garantendo la conformità legale in modo continuo.